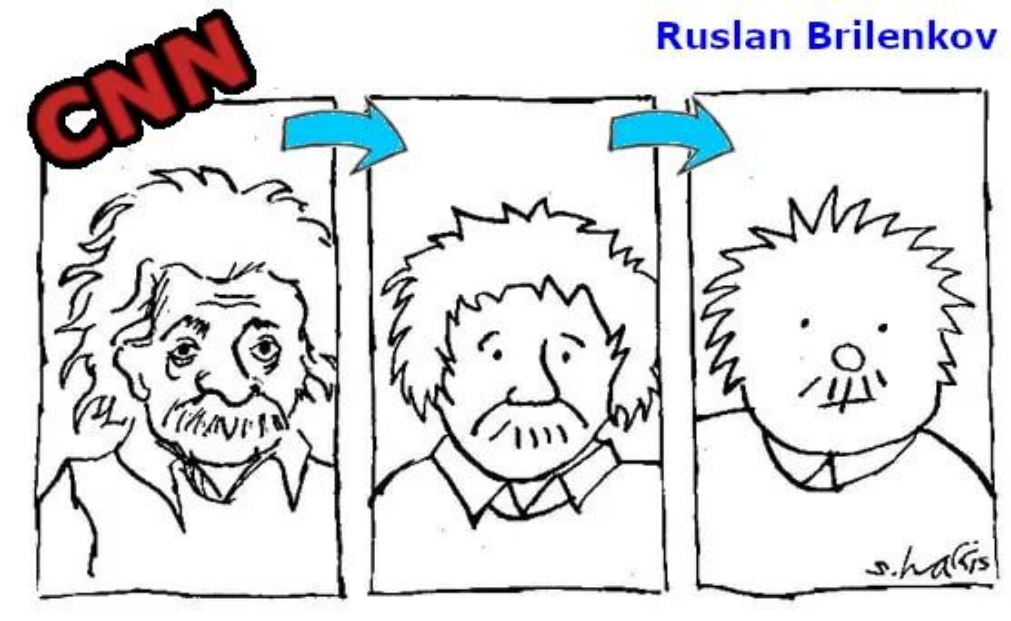
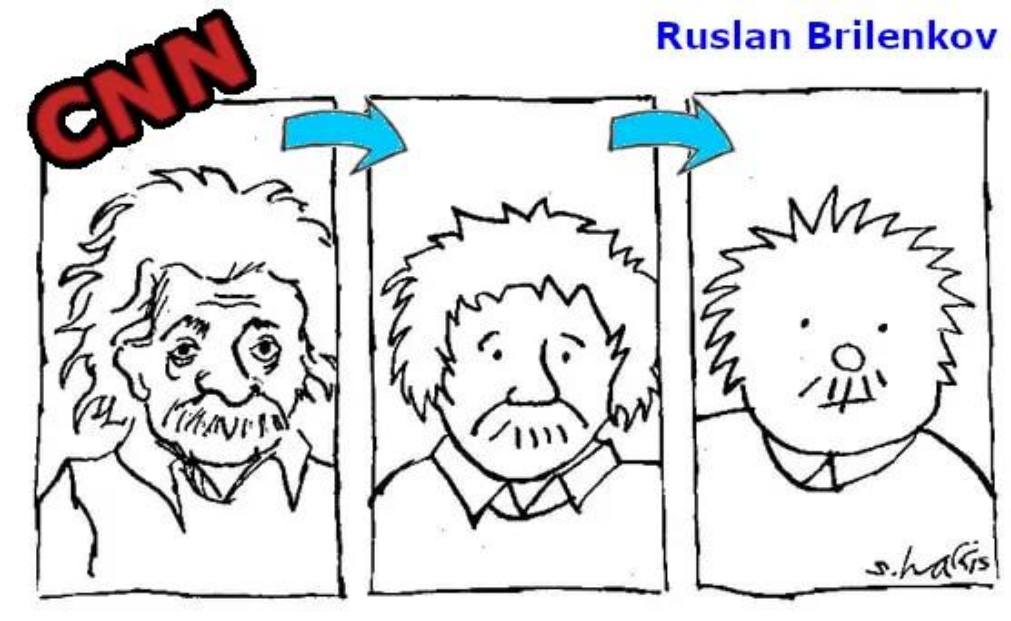
Discussion Module 10



Outline:

- Project Proposal/Checkpoint
- Project Presentation Schedule
- GAN & VAR
- Forward Part *Spelled Out*
- Backward Part *Spelled Out*

Discussion Module 10



Outline:

- **Project Proposal/Checkpoint**
- Project Presentation Schedule
- GAN & VAR
- Forward Part *Spelled Out*
- Backward Part *Spelled Out*

Project Proposal/Checkpoint?

- Final Project Checkpoint, due **Apr 7th**

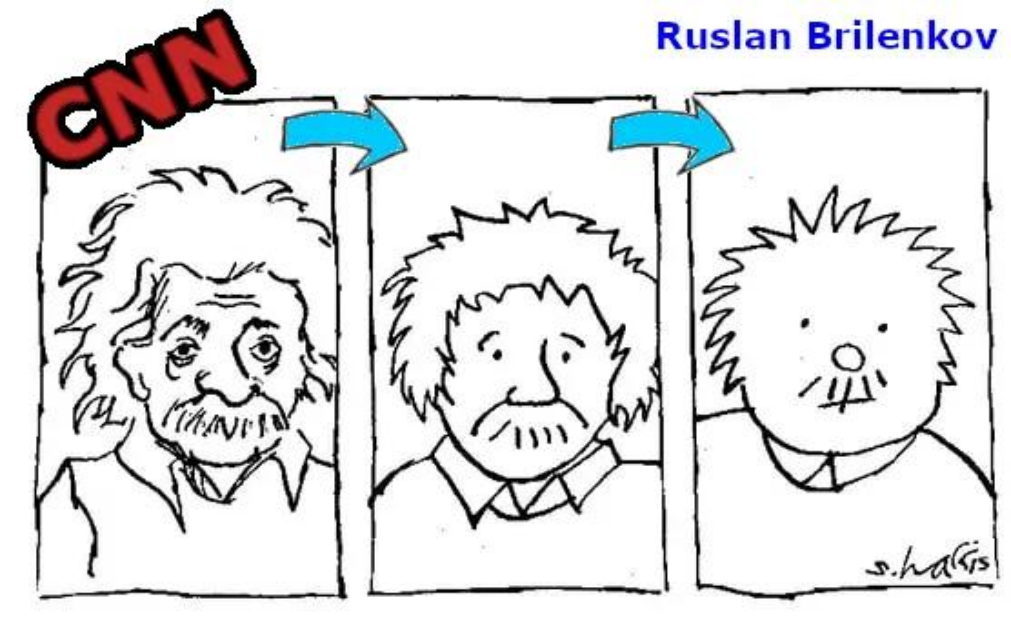
Please submit submit **one assignment per group** and add all of group members your project checkpoint to GradeScope.

- one page
- project title
- team members
- objective and goal of the project
- EDA results
- outline of the project
- references

- Feedback on Proposal (GradeScope)

- Questions: Office Hours Spencer and/or Markus

Discussion Module 10



Outline:

- Project Proposal/Checkpoint
- **Project Presentation Schedule**
- GAN & VAR
- Forward Part *Spelled Out*
- Backward Part *Spelled Out*

Project Presentation Schedule

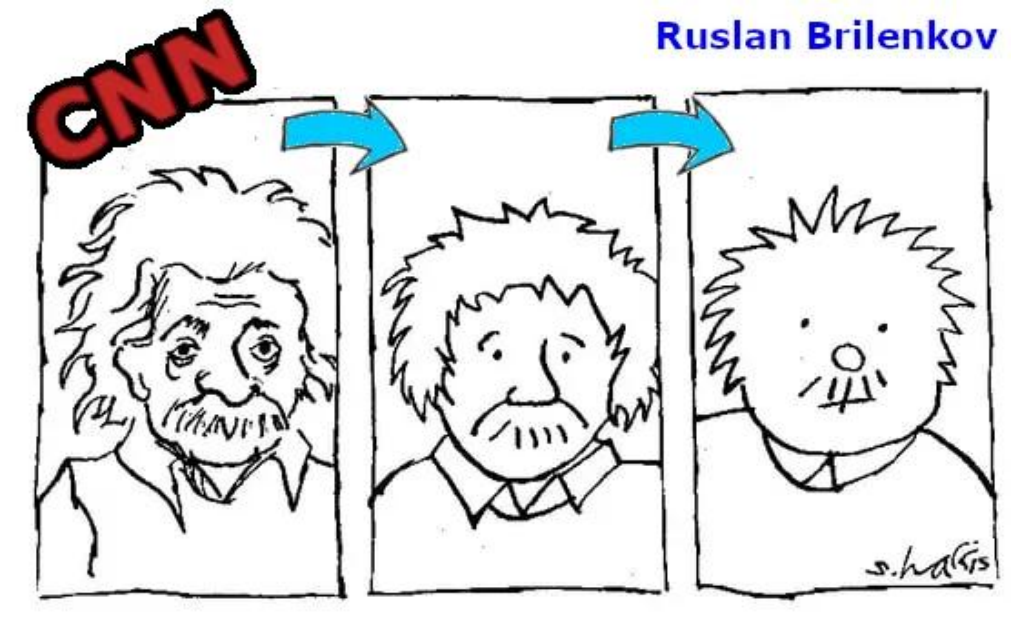
5:30 – 7:30pm, 20min + 10min Q&A, i. e. max three teams per session

Monday Apr 20th:

Thursday Apr 23rd:

Friday Apr 24th:

Discussion Module 10



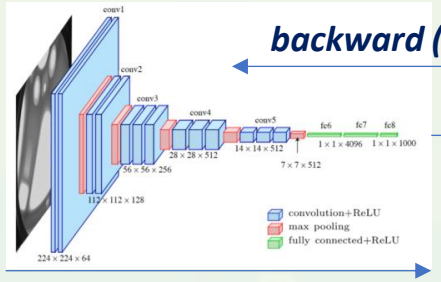
Outline:

- Project Proposal/Checkpoint
- Project Presentation Schedule
- **GAN & VAR**
- Forward Part *Spelled Out*
- Backward Part *Spelled Out*

Discussion Module 10

Generative Adversarial Network (GAN)

set of real images

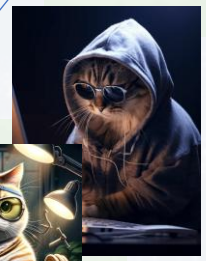


backward (up-sampling)

forward (down-sampling)

discriminator

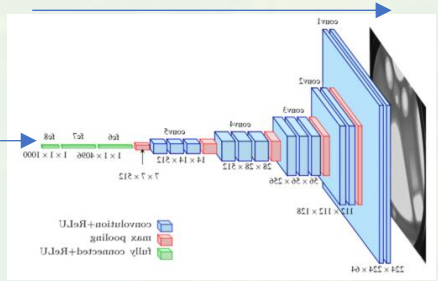
binary classification:
fake or real
(loss: cross entropy)



set of fake images

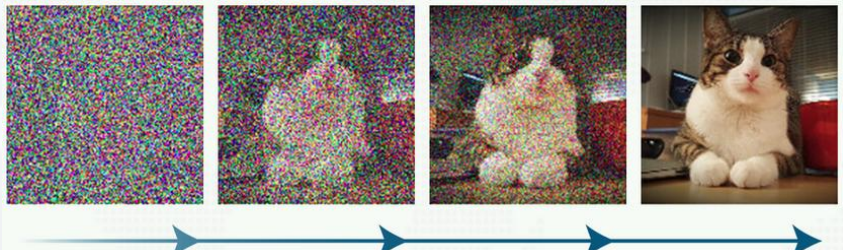
forward (up-sampling)

random vector



backward (down-sampling)

generator

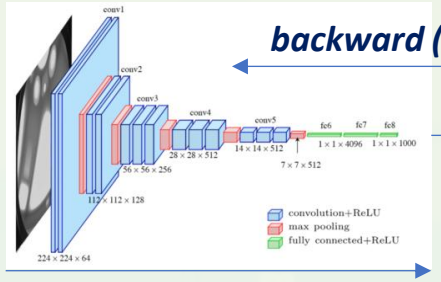


yes: The vector for the generator is random. We turn noise into cats!

Discussion Module 10

Generative Adversarial Network (GAN)

set of real images



backward (up-sampling)

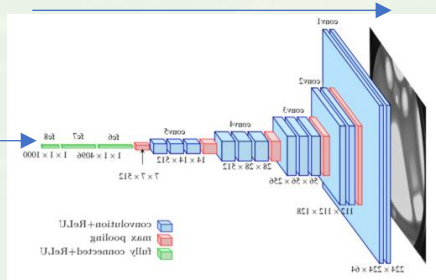
forward (down-sampling)

binary classification:
fake or real
(loss: cross entropy)

set of fake images

random vector

forward (up-sampling)



backward

generator

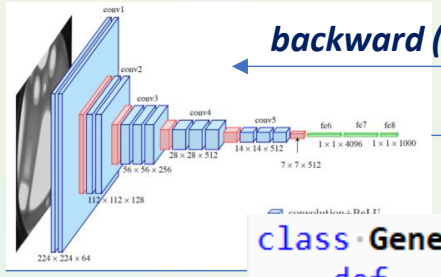
```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
        nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),
        nn.LeakyReLU(0.2),
        nn.Dropout(0.25),
        nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
        nn.ZeroPad2d((0, 1, 0, 1)),
        nn.BatchNorm2d(64, momentum=0.82),
        nn.LeakyReLU(0.25),
        nn.Dropout(0.25),
        nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
        nn.BatchNorm2d(128, momentum=0.82),
        nn.LeakyReLU(0.2),
        nn.Dropout(0.25),
        nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256, momentum=0.8),
        nn.LeakyReLU(0.25),
        nn.Dropout(0.25),
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Sigmoid()
```

standard classification CNN

Discussion Module 10

Generative Adversarial Network (GAN)

set of real images



backward (up-sampling)

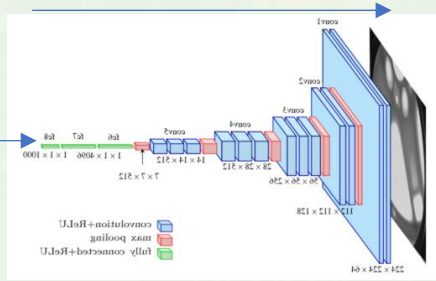
forward (down-

binary classification:
fake or real
(loss: cross entropy)

set of fake images

random vector

forward (up-sampling)



generator

```
class Generator(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()

        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128 * 8 * 8),
            nn.ReLU(),
            nn.Unflatten(1, (128, 8, 8)),
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128, momentum=0.78),
            nn.ReLU(),
            nn.Upsample(scale_factor=2),
            nn.Conv2d(128, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64, momentum=0.78),
            nn.ReLU(),
            nn.Conv2d(64, 3, kernel_size=3, padding=1),
            nn.Tanh()
        )

    def forward(self, z):
        img = self.model(z)
        return img
```

up-sampling:
like in standard segmentation CNN

Discussion Module 10

Generative Adversarial Network (GAN)

x: real image
z: latent vector (random numbers)

$p_D(x)$: probability that discriminator *D* classifies *x* as real image
$G(z)$: fake image generated from generator *G*

$\mathcal{L}_D$: loss function of discriminator *D*
$\mathcal{L}_G$: loss function of generator *G*

(mean) binary cross entropy (BCE): $\mathcal{L}_D = -\frac{1}{N}\sum_{i=1}^{N}\log[p_D(x_i)] + \log[1 - p_D(G(z_i))]$

over all *N* images $\mathcal{L}_G = \frac{1}{N}\sum_{i=1}^{N}\log[1 - p_D(G(z_i))]$

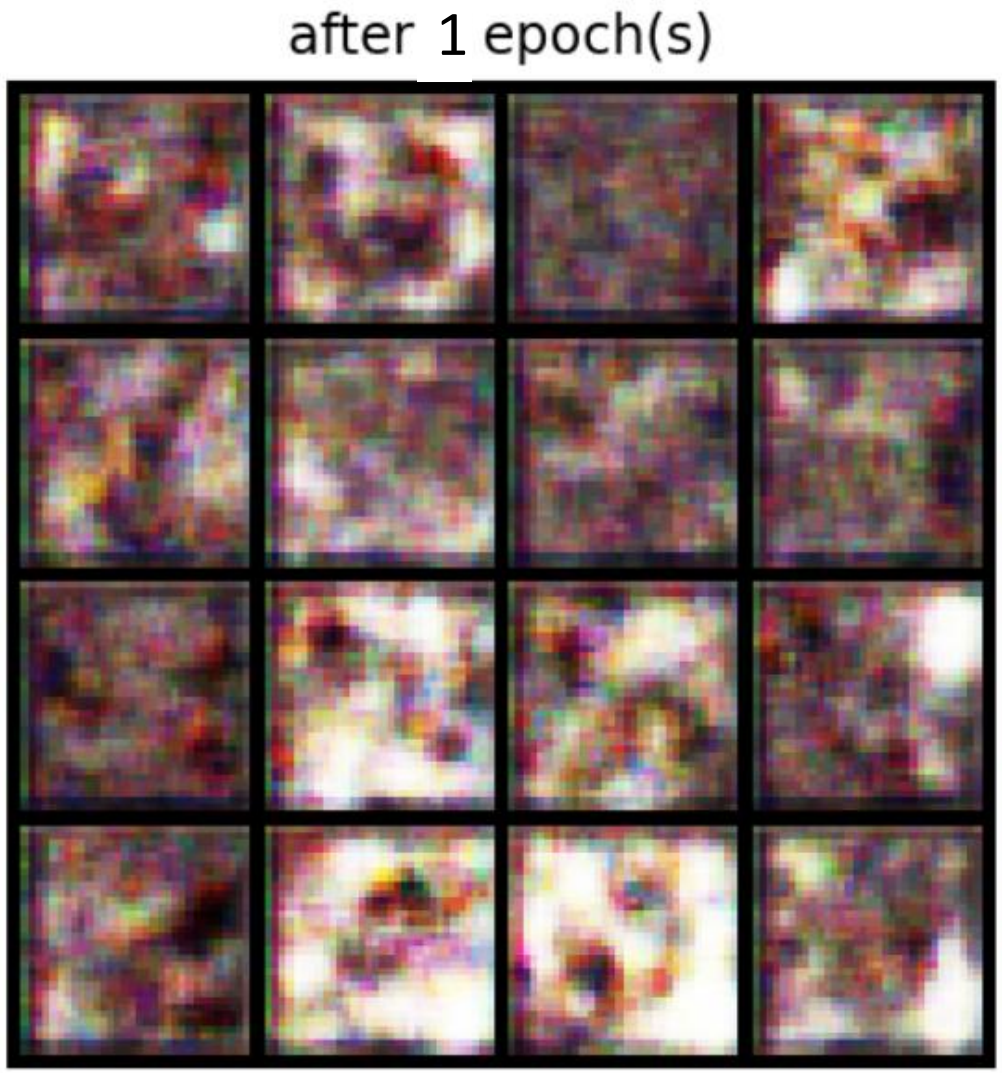
ideal case for *D*: $\mathcal{L}_D = -\log[1] - \log[1 - 0] = 0$
worst case for *D*: $\mathcal{L}_D = -\log[1] - \log[1 - \mathbf{0.5}] = \mathbf{0.69}$

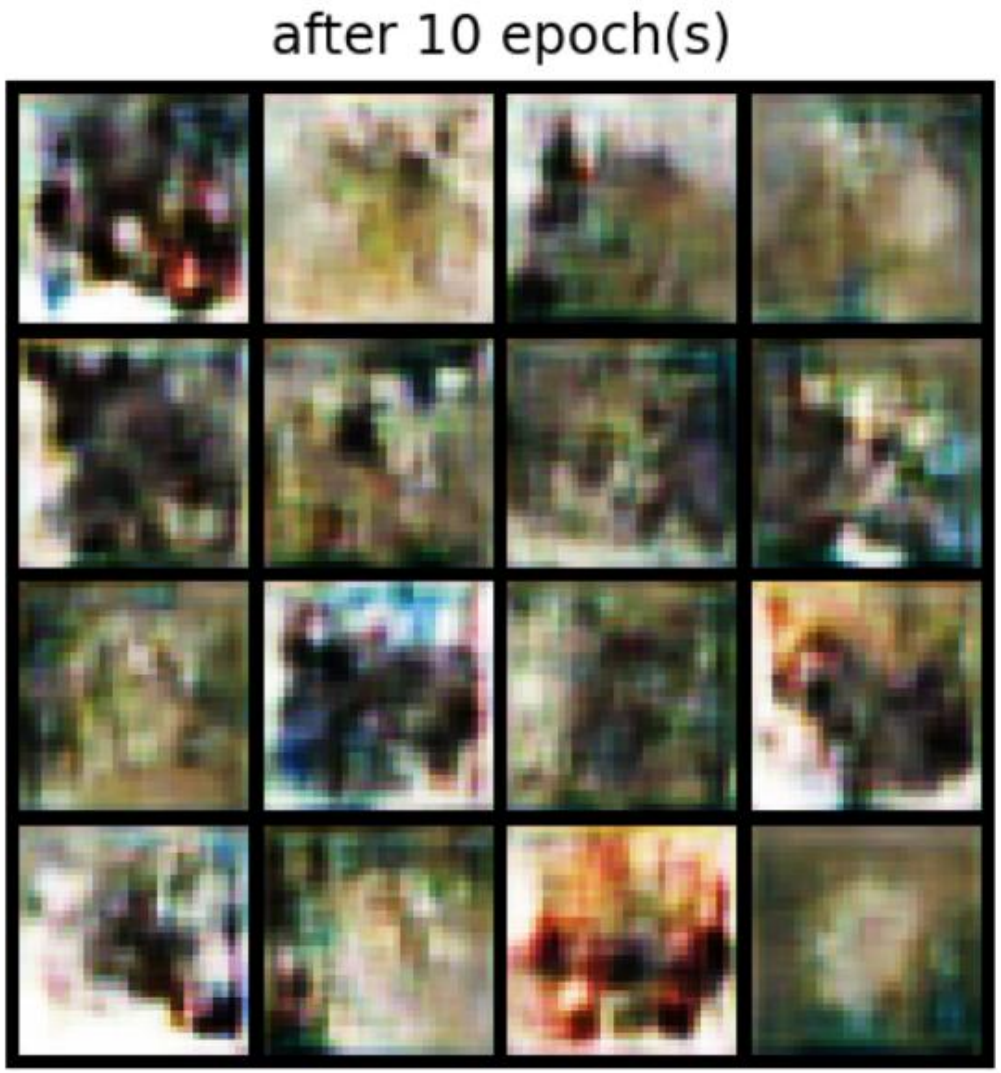
ideal case for *G*: $\mathcal{L}_G = \log[1 - \mathbf{0.5}] = \mathbf{-0.69}$
worst case for *G*: $\mathcal{L}_G = \log[1 - 0] = 0$

Discussion Module 10

Generative Adversarial Network (GAN)

check out `GAN architecture.ipynb`

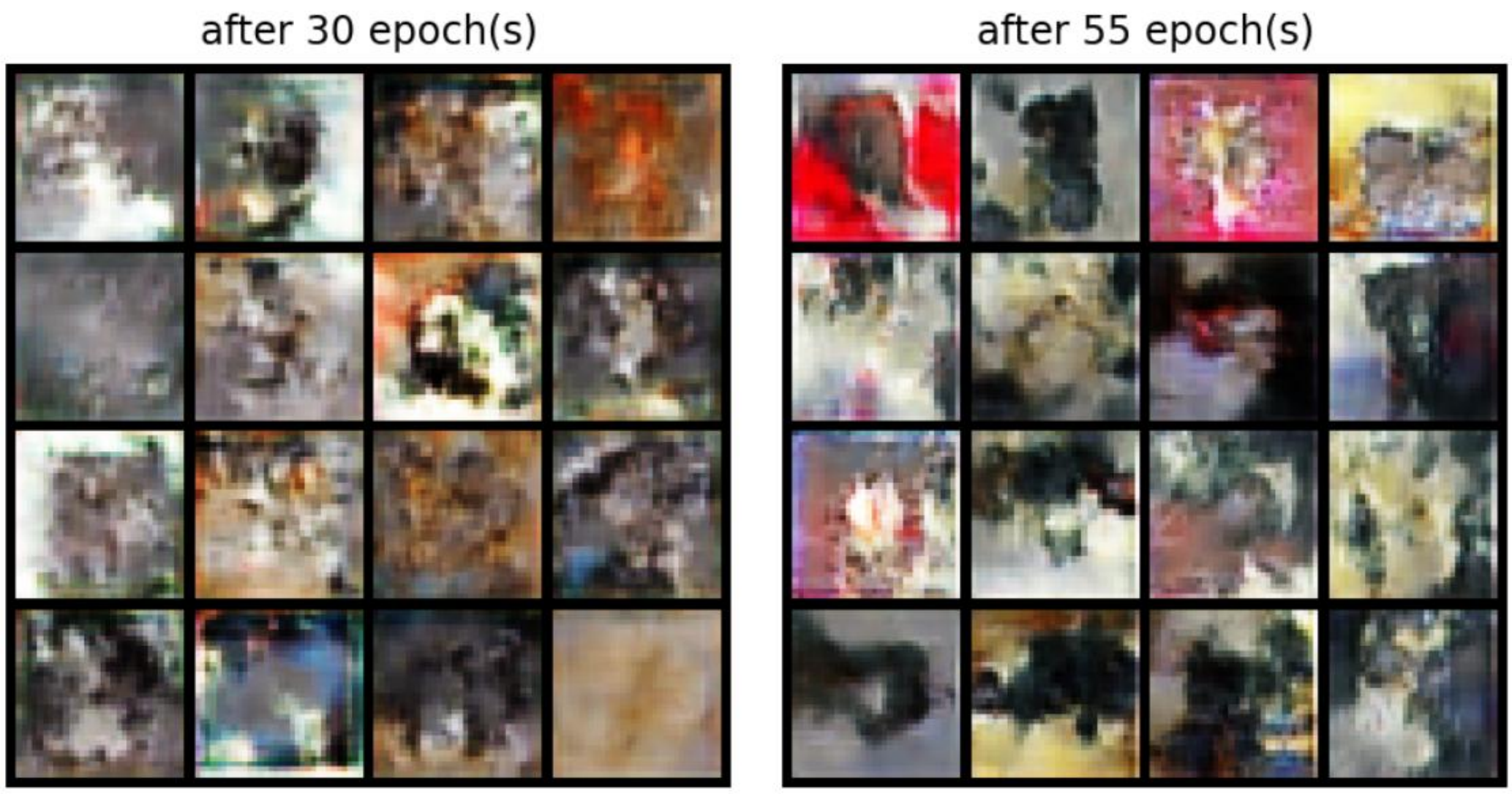




Discussion Module 10

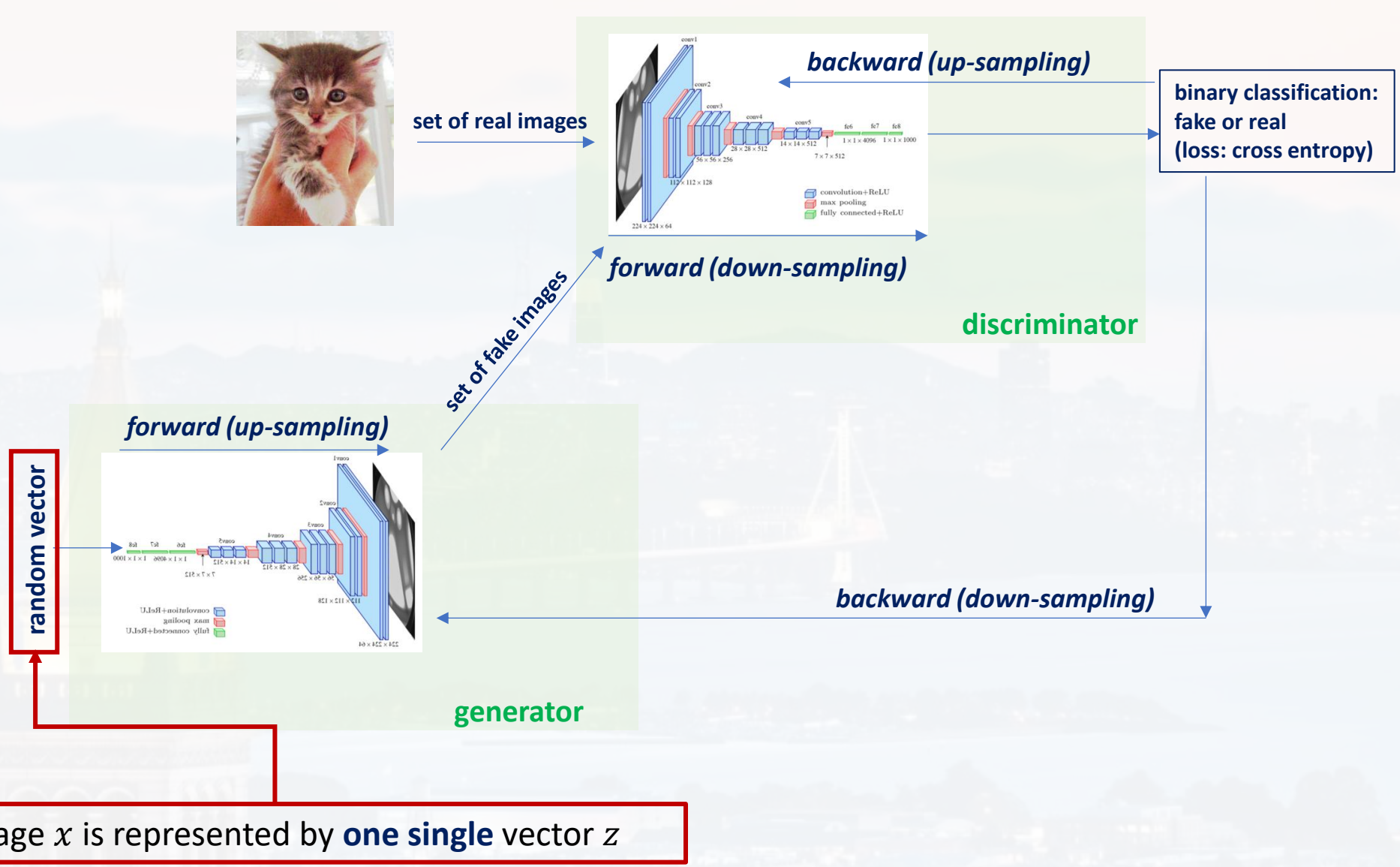
Generative Adversarial Network (GAN)

check out `GAN architecture.ipynb`



Discussion Module 10

GAN:

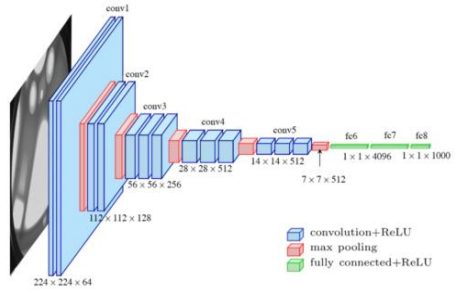


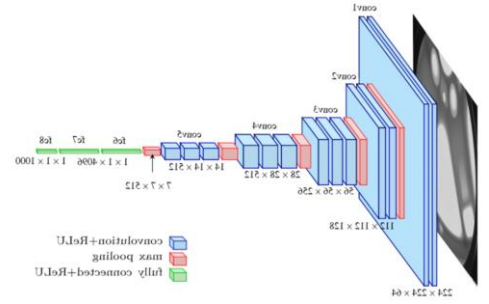
each image x is represented by **one single** vector z

Discussion Module 10

Variational AutoEncoder (VAE)

now:

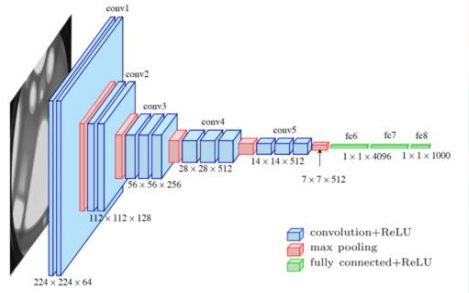


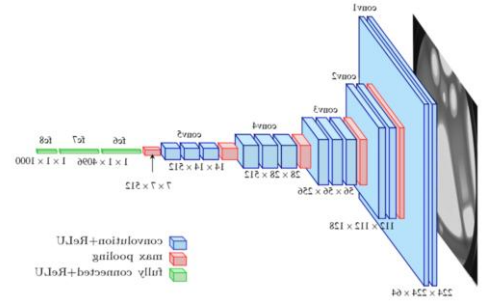


Discussion Module 10

Variational AutoEncoder (VAE)

now:





GAN: each image x is represented by **one single** vector z

VAE: each image x is represented by **a distribution** we sample from in order **to get z**

i: index over dimensions of z

$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ (max entropy!)

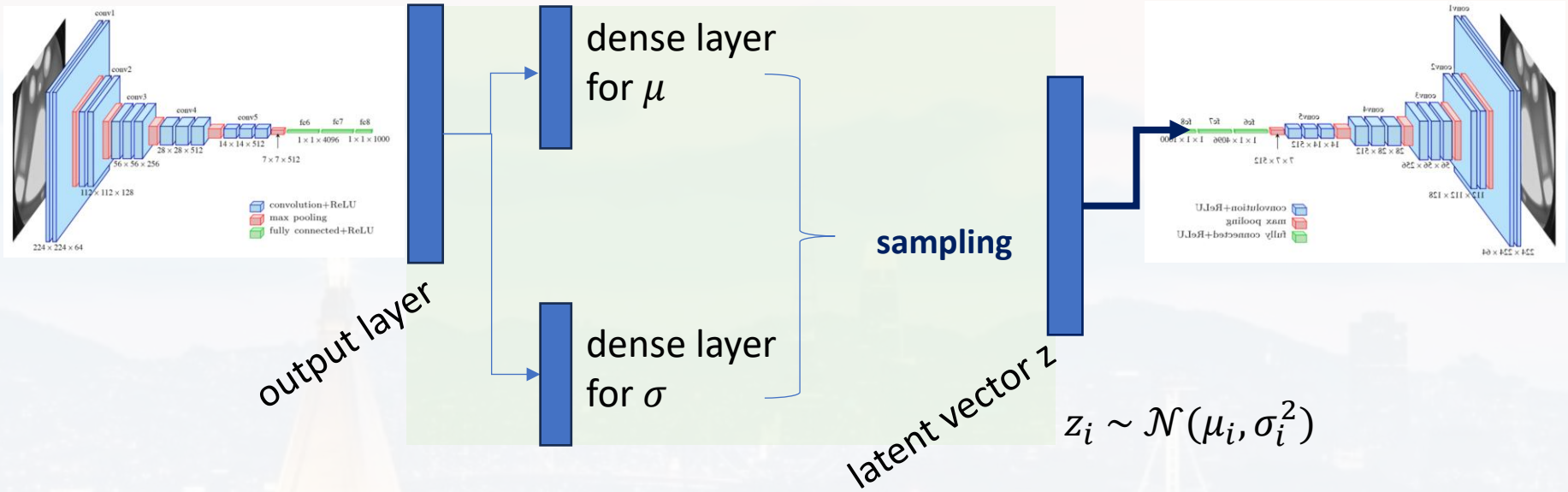
→ generate image from up-sampling as before

→ randomnes accounts for diverisity in generated data

Discussion Module 10

Variational AutoEncoder (VAE)

now:



$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$

example: output layer has shape (None, N)

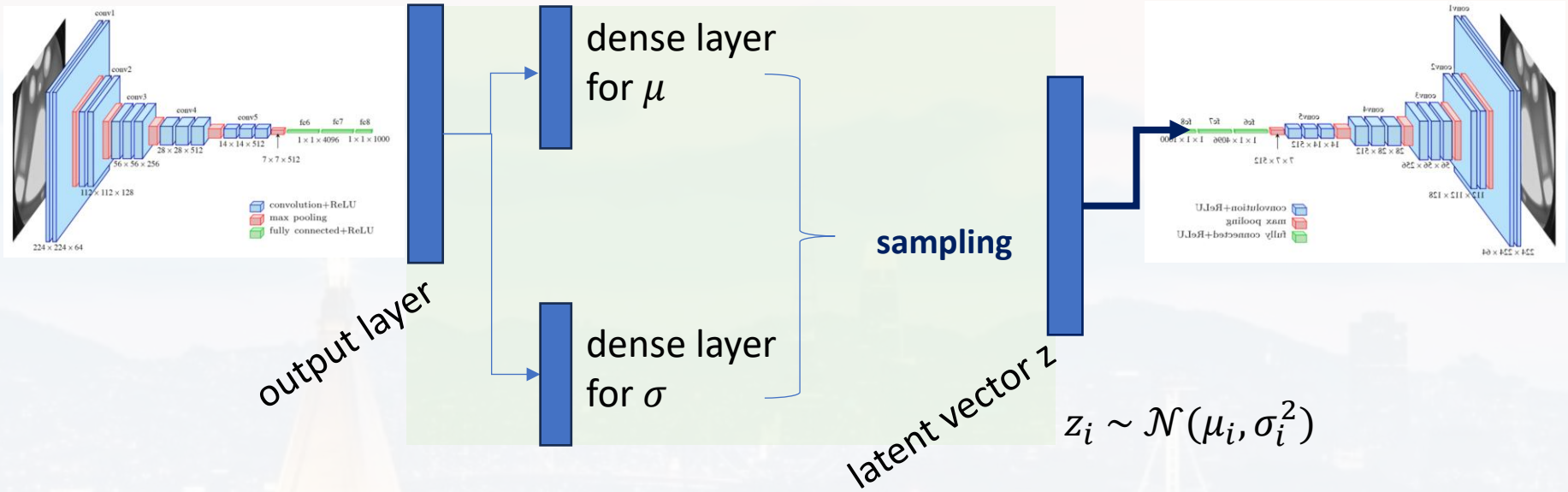
we want M dimensional encoding for z

→ weights in dense layer for σ and μ have shape (N, M)

→ latent vector has length M

Variational AutoEncoder (VAE)

now:



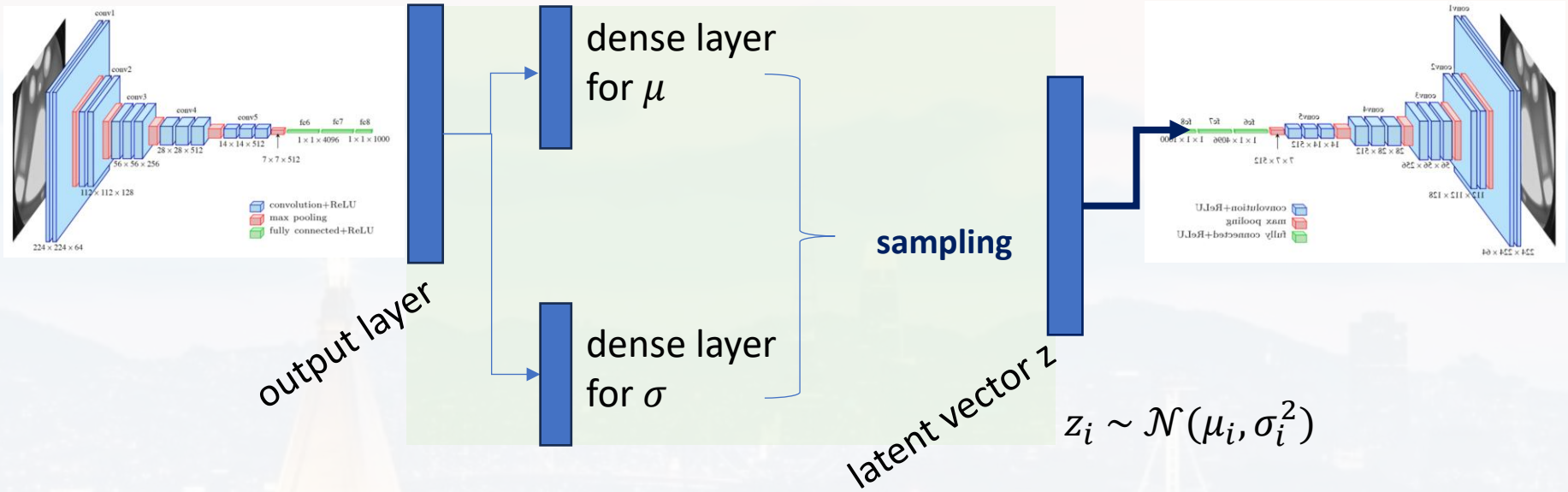
backprop:

- μ_i and σ_i^2 have been generated deterministically from a dense layer
- we need to run backpropagation through μ_i and σ_i^2 in order to adjust the weights/biases
- in order to keep randomness, we actually sample via

$z_i = \mu_i + \sigma_i \cdot \epsilon$ where $\epsilon \sim \mathcal{N}(0,1)$ is not affected by backpropagation

Variational AutoEncoder (VAE)

now:



loss: 1) **reconstruction loss L_R**, i. e. the difference between input image x and reconstructed image $\boldsymbol{p(x|z)}$. Note: L_R alone would **lead to overfitting**, exact image gets reproduced!

2) we need a **regularization term** that counteracts $\boldsymbol{L_R}$:

KL-divergence $\boldsymbol{L_{KL} = KL[q(z|x)|p(z)]}$, which increases if learned distribution $\boldsymbol{q(z|x)}$ gets too far from prior $\boldsymbol{p(z)} \sim \mathcal{N}(0,1)$

Variational AutoEncoder (VAE)

loss: 1) **reconstruction loss L_R**, i. e. the difference between input image x and reconstructed image $p(x|z)$. Note: L_R alone would **lead to overfitting**, exact image gets reproduced!

2) we need a **regularization term** that counteracts L_R:

KL-divergence $L_{KL} = KL[q(z|x)|p(z)]$, which increases if learned distribution $q(z|x)$ gets too far from prior $p(z) \sim \mathcal{N}(0,1)$

actual distribution $p(x)$; some other distribution $q(x)$

How much is our information "off" if we work with the approximation $q(x)$?

$$-\int p(x)\log[q(x)]\,dx - \left[-\int p(x)\log[p(x)]\,dx\right] = -\int p(x)\log\left[\frac{q(x)}{p(x)}\right]dx = KL(p||q)$$

KL or ***Kullback-Leiber divergence***

It is **not** a distance! $KL(p||q) \neq KL(q||p)$

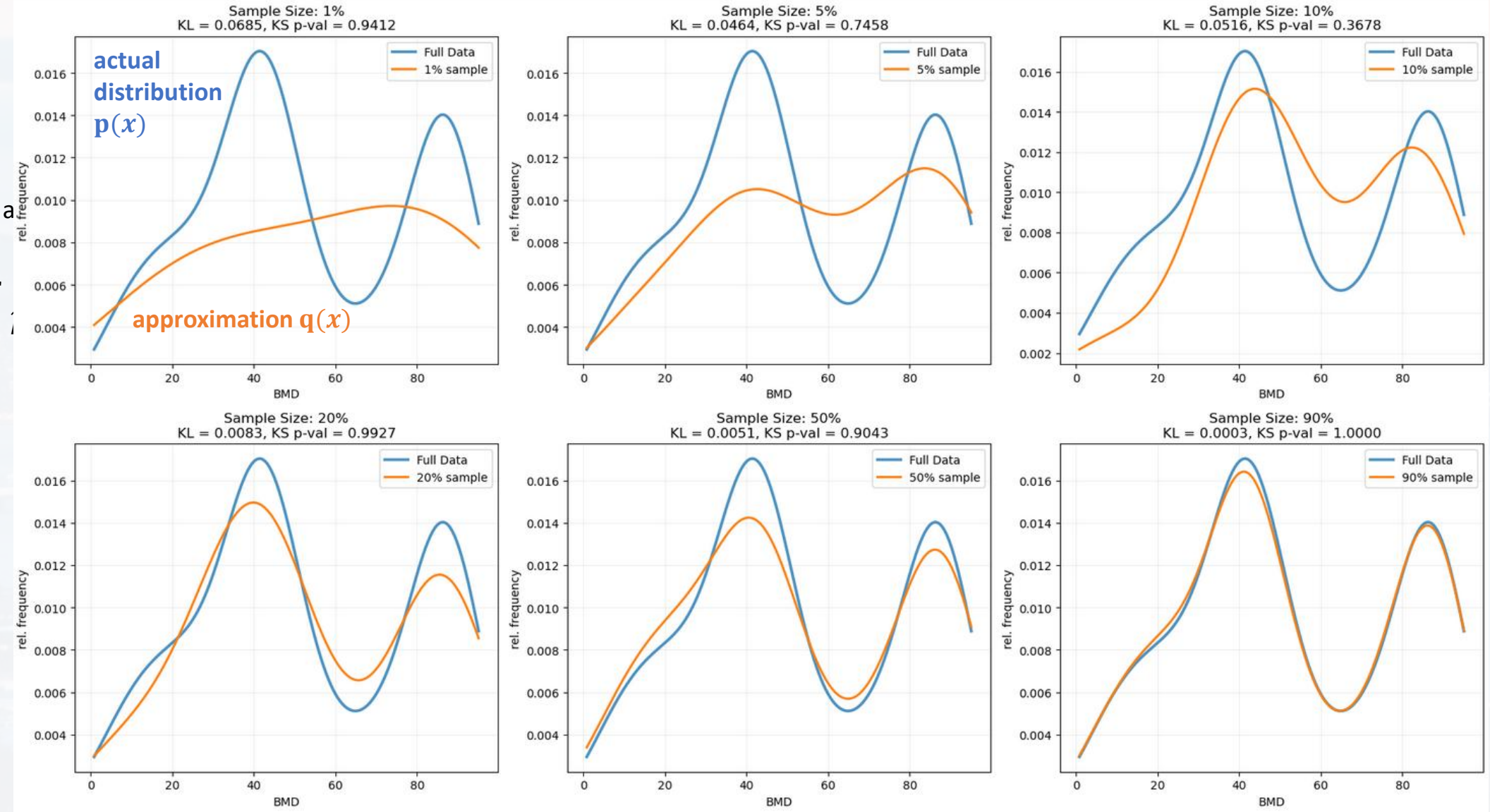
Discussion Module 10

Variational AutoEncoder (VAE)

loss: 1) **reconstruction loss L_R**, i. e. the difference between input image x and reconstructed image $p(x|z)$. Note: L_R alone would **lead to overfitting**, exact image gets reproduced!

a

$$KL(p||q) = -\int$$



Variational AutoEncoder (VAE)

loss: 1) **reconstruction loss L_R**, i. e. the difference between input image x and reconstructed image $p(x|z)$. Note: L_R alone would **lead to overfitting**, exact image gets reproduced!

2) we need a **regularization term** that counteracts L_R:

KL-divergence $L_{KL} = KL[q(z|x)|p(z)]$, which increases if learned distribution $q(z|x)$ gets too far from prior $p(z) \sim \mathcal{N}(0,1)$

see here for more details

check out `VAR architecture.ipynb`

Discussion Module 10

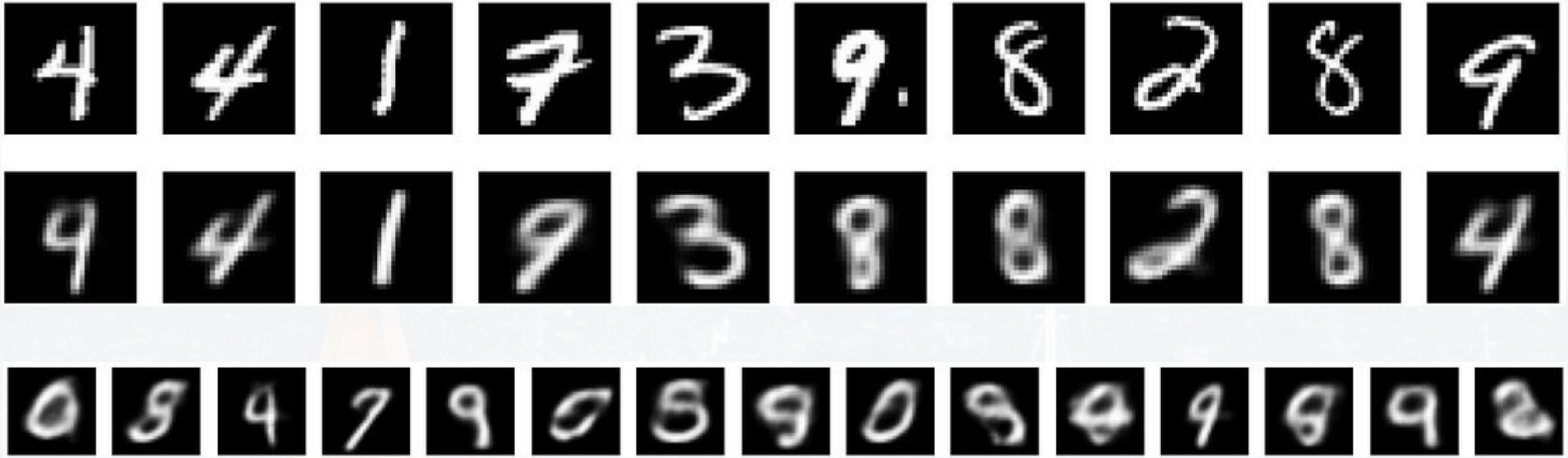
Variational AutoEncoder (VAE)

check out `VAR architecture.ipynb`

original

reconstruction

generated



Discussion Module 10

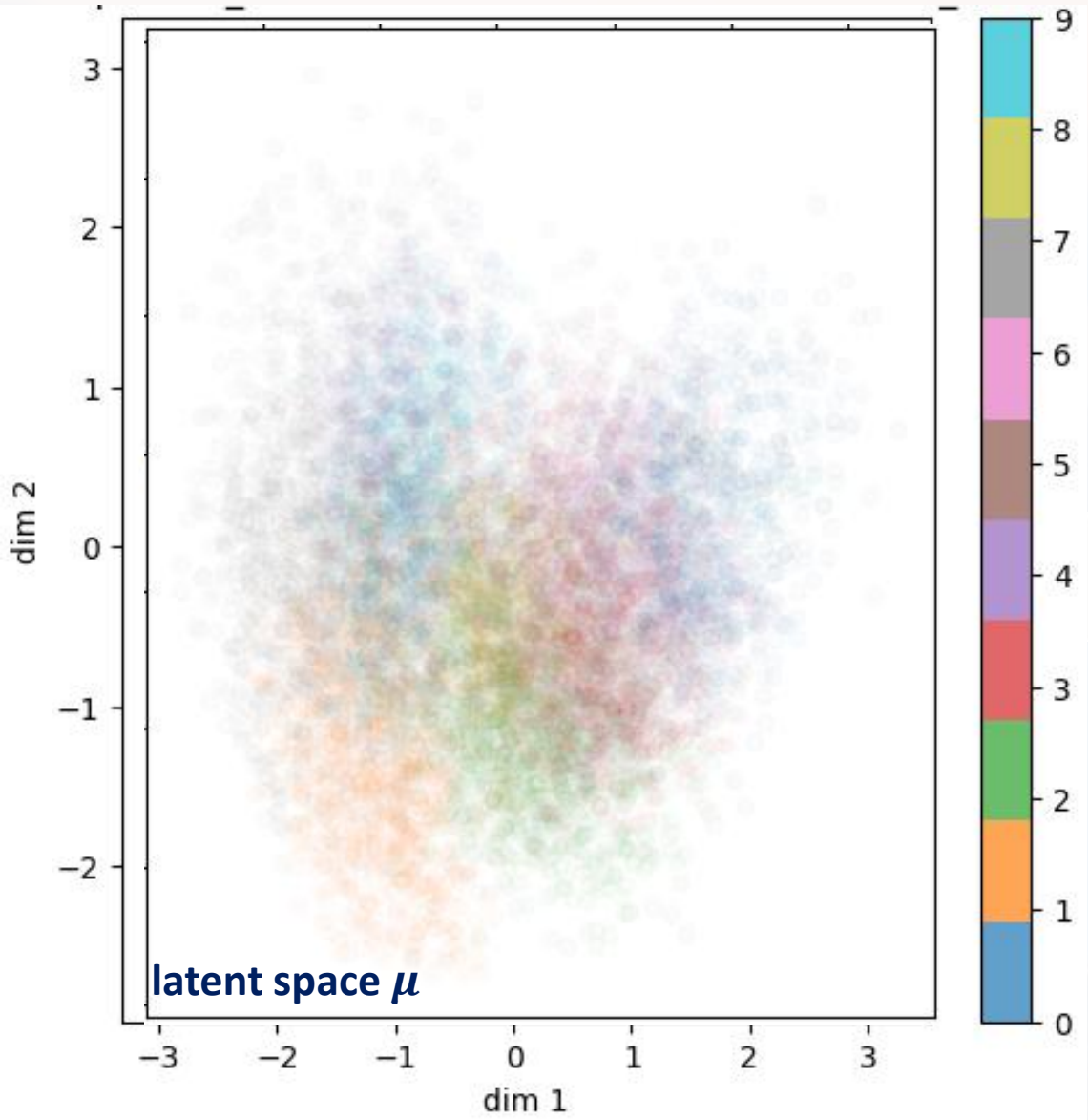
Variational AutoEncoder (VAE)

check out `VAR architecture.ipynb`

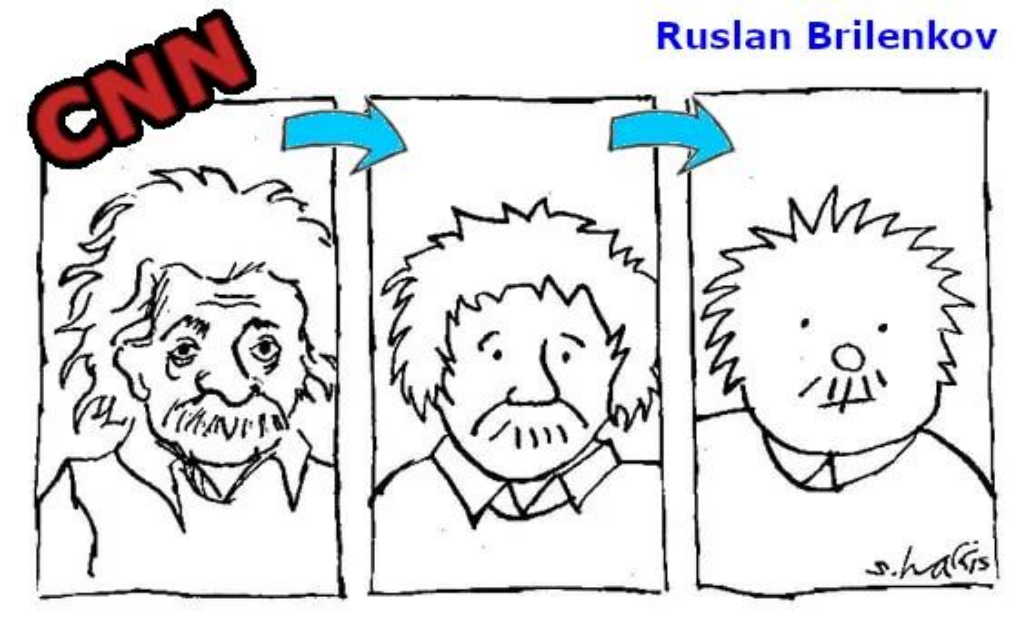
original

reconstruction

generated



Discussion Module 10



Outline:

- Project Proposal/Checkpoint
- Project Presentation Schedule
- GAN & VAR
- **Forward Part *Spelled Out***
- Backward Part *Spelled Out*

see MyCNN.py

- MyCNN.py
  - Read_Scale_Imds
  - ConvLayer
    - __init__
    - forward
    - backward
  - Average_Pool
  - Min_Pool
  - Max_Pool
  - Sigmoid
  - Tanh
  - Activation_ReLU
  - Flat
  - Layer_Dense
  - Activation_Softmax
  - Loss
  - Loss_CategoricalCrossEntropy
  - CalcSoftmaxLossGrad
  - Optimizer_SGD

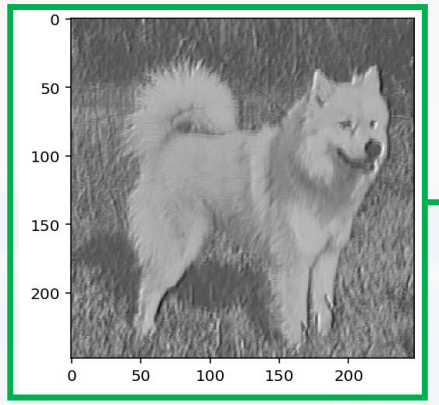
we know these parts already

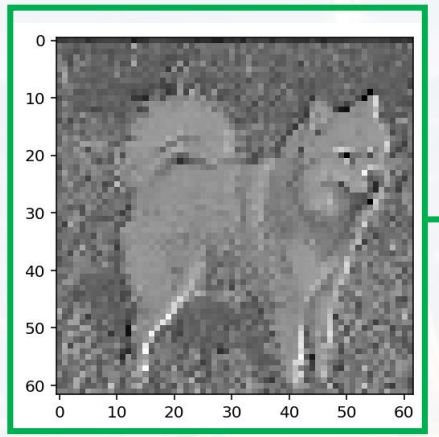
```
import matplotlib.pyplot as plt
from MyCNN import *

read_and_scale = Read_Scale_Imds(5, [250,250])
[I, _]         = read_and_scale.Read_Scale()

Conv1 = ConvLayer(3,3,5)
Conv2 = ConvLayer(5,5,4)
Conv3 = ConvLayer(2,2,4)

Conv1.forward(I,0,1)
Conv2.forward(Conv1.output,2,4)
Conv3.forward(Conv2.output,0,2)

plt.imshow(Conv1.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(Conv2.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(Conv3.output[:,:,0,4], cmap = 'gray')
plt.show()
```

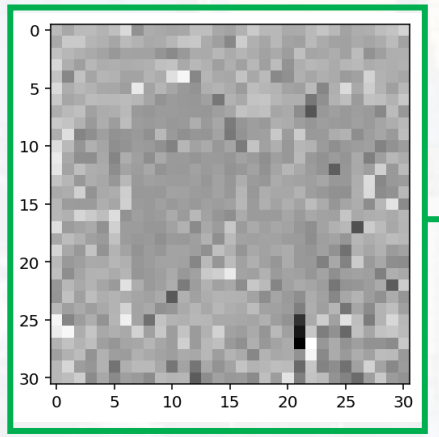
initializing convolution layer (kernel size x kernel size, number of kernels)

passing image through convolution layer (padding, stride lenght)

Forward Part Spelled Out








```
import matplotlib.pyplot as plt
from MyCNN import *

read_and_scale = Read_Scale_Imds(5, [250,250])
[I, _]         = read_and_scale.Read_Scale()

Conv1 = ConvLayer(3,3,5)
Conv2 = ConvLayer(5,5,4)
Conv3 = ConvLayer(2,2,4)

Conv1.forward(I,0,1)
Conv2.forward(Conv1.output,2,4)
Conv3.forward(Conv2.output,0,2)
```

```
plt.imshow(Conv1.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(Conv2.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(Conv3.output[:,:,0,4], cmap = 'gray')
plt.show()
```

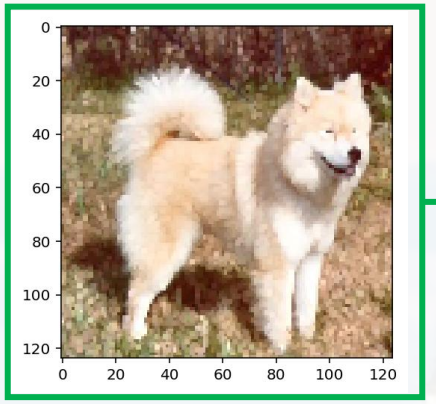
see MyCNN.py

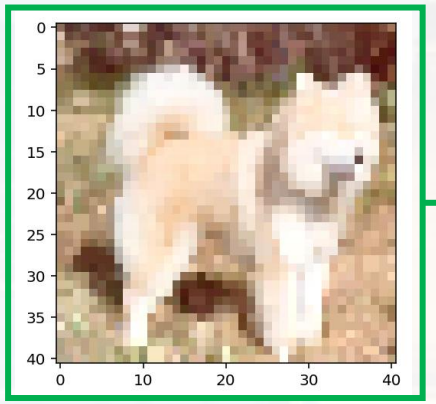
- MyCNN.py
  - Read_Scale_Imds
  - ConvLayer
    - __init__
    - forward
    - backward
  - Average_Pool
  - Min_Pool
  - Max_Pool
  - Sigmoid
  - Tanh
  - Activation_ReLU
  - Flat
  - Layer_Dense
  - Activation_Softmax
  - Loss
  - Loss_CategoricalCrossEntropy
  - CalcSoftmaxLossGrad
  - Optimizer_SGD

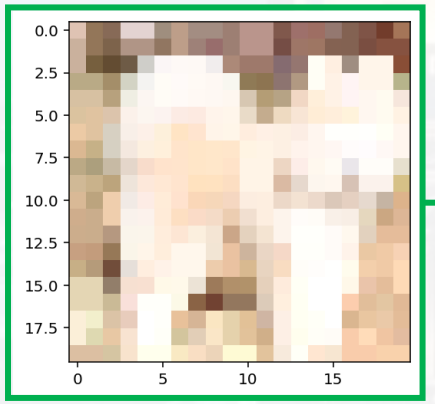
```
y_start = y*stride
y_end   = y_start + yK
x_start = x*stride
x_end   = x_start + xK

sx = slice(x_start, x_end)
sy = slice(y_start, y_end)

current_slice     = currentIm_pad[sx,sy,c]
slice_max         = float(current_slice.max())
output[x, y, c, i] = slice_max
```

here: max pool

Forward Part Spelled Out








```
MP1    = Max_Pool()
MP2    = Max_Pool()
MP3    = Max_Pool()
```

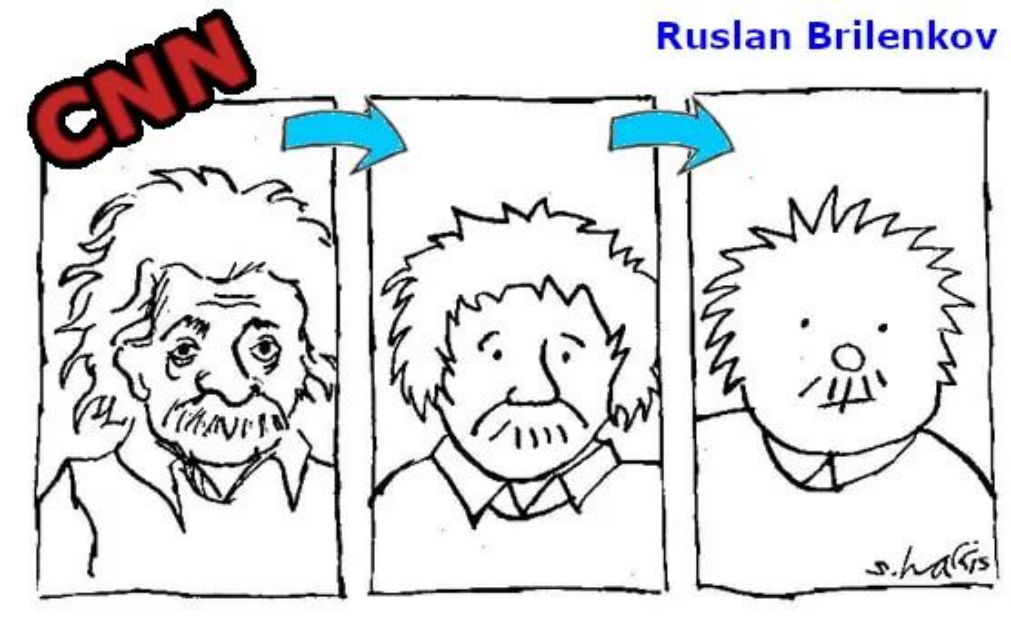
initializing max pool layer

```
MP1.forward(I,2,3)
MP2.forward(MP1.output,3,3)
MP3.forward(MP2.output,2,3)
```

passing image through max pool layer (stride length, kernel size)

```
plt.imshow(MP1.output[:,:,:,4].astype(int))
plt.show()
plt.imshow(MP2.output[:,:,:,4].astype(int))
plt.show()
plt.imshow(MP3.output[:,:,:,4].astype(int))
plt.show()
```

Discussion Module 10



Outline:

- Project Proposal/Checkpoint
- Project Presentation Schedule
- GAN & VAR
- Forward Part *Spelled Out*
- **Backward Part *Spelled Out***

last time for dense layer:

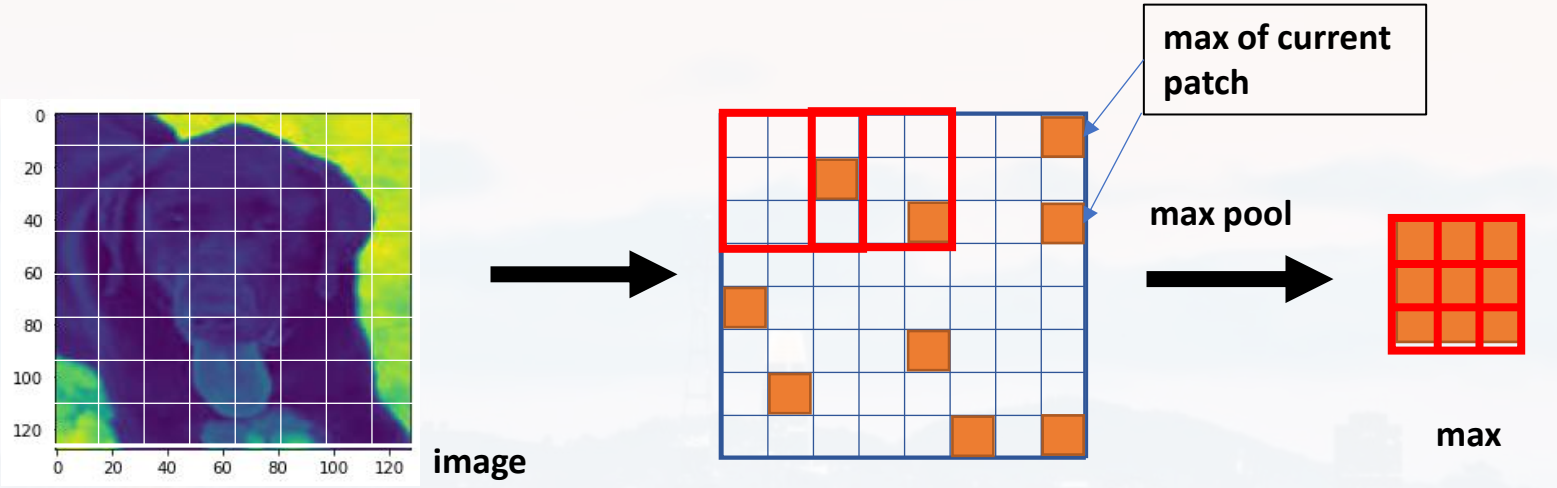
```
class Layer_Dense:

        def __init__(self, n_inputs, n_neurons):
                self.weights = np.random.rand(n_inputs, n_neurons)
                self.biases  = np.zeros((1, n_neurons))

        def forward(self, inputs):

                self.output  = np.dot(inputs, self.weights) + self.biases
                self.inputs  = inputs

        def backward(self, dvalues):

                self.dweights  = np.dot(self.inputs.T, dvalues)
                self.dinputs   = np.dot(dvalues, self.weights.T)
                self.dbiases   = np.sum(dvalues, axis = 0, keepdims = True)
```

outer derivative

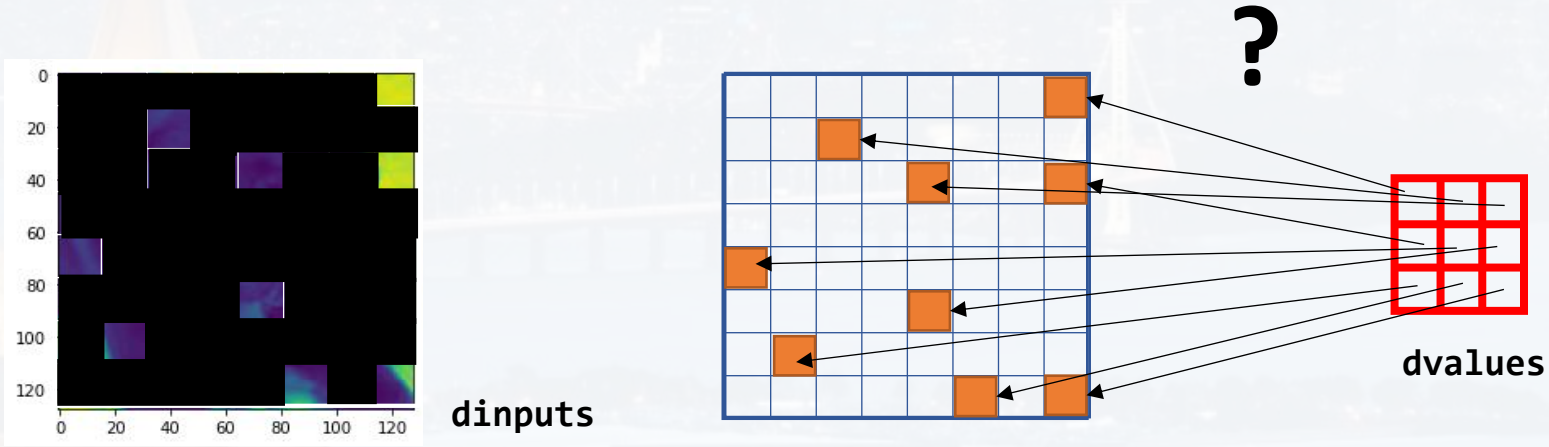
max pool

for **backpropagation**, we need to **up-sample** the images!

forward:



backward:



We need **to track, where the max came from** for each patch: creating a **mask** in the forward part

For those pixel: `dinputs = dvalues`

Discussion Module 10

Backward Part Spelled Out

max pool

for **backpropagation**, we need to **up-sample** the images!

Input I stride = 3; xK = yK = 5

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	538	538	538	538	541	541
2	0	538	538	538	538	541	541
3	0	535	535	538	538	541	541
4	0	535	535	538	538	541	541
5	0	535	535	538	538	538	538
6	0	535	535	538	538	538	538
7	0	535	535	538	538	541	541
8	0	535	535	538	538	541	541
9	0	535	535	538	538	538	544

max pool

output

	0	1	2	3	4	5	6
0	538	544	545	551	543	357	364
1	538	544	547	550	544	369	361
2	538	544	548	554	554	369	361
3	544	547	548	554	554	361	358
4	544	544	499	364	362	368	375
5	424	363	355	355	350	363	375
6	352	354	358	358	356	356	357
7	352	353	354	359	368	368	350
8	353	353	362	362	360	359	356
9	357	356	367	367	366	365	356

mask

and so on…

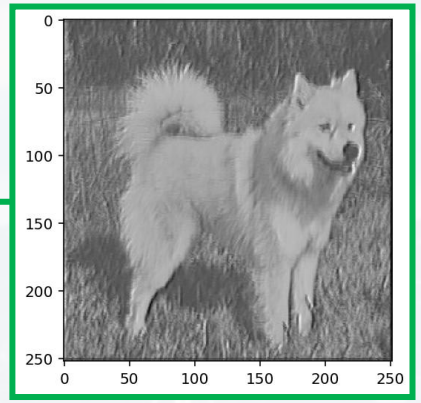
	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	538	538	538	538	0	0
2	0	538	538	538	538	0	0
3	0	0	0	538	538	0	0
4	0	0	0	538	538	0	0
5	0	0	0	538	538	0	0
6	0	0	0	538	538	0	0
7	0	0	0	538	538	0	0
8	0	0	0	538	538	0	0
9	0	0	0	538	538	0	544

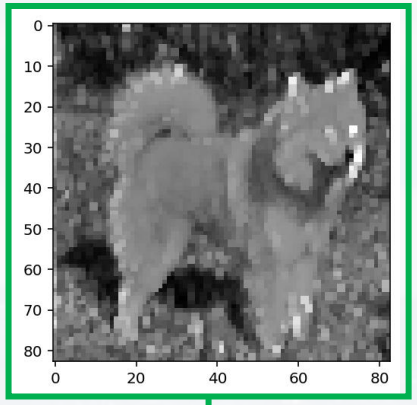
max pool

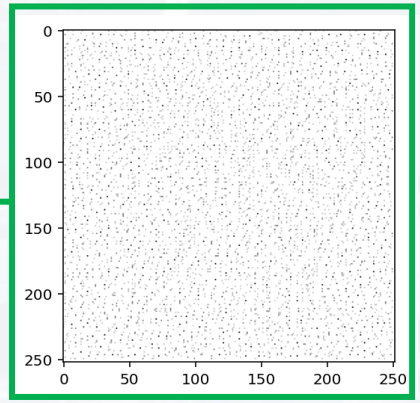
for **backpropagation**, we need to **up-sample** the images!

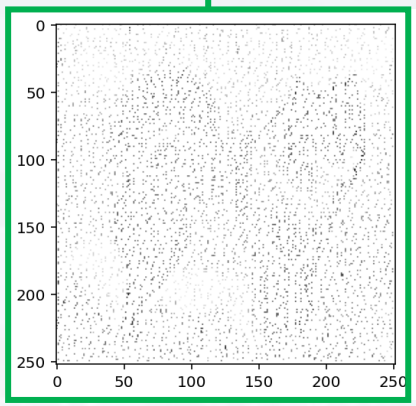
`dinputs` and mask have to look the same (but different values)!

```
Conv1.forward(I,2,1)
MP1.forward(Conv1.output,3,5)
MP1.backward(MP1.output)

plt.imshow(Conv1.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(MP1.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(MP1.dinputs[:,:,0,4], cmap = 'gray_r')
plt.show()
plt.imshow(MP1.mask[:,:,0,4], cmap = 'gray_r')
plt.show()

Diff = MP1.mask[:,:,0,4]/np.max(MP1.mask[:,:,0,4])-\
       MP1.dinputs[:,:,0,4]/np.max(MP1.dinputs[:,:,0,4])
```









max pool

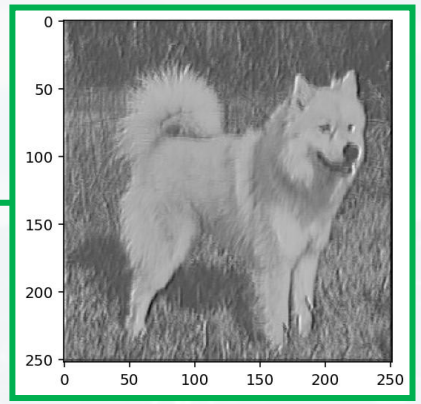
for **backpropagation**, we need to **up-sample** the images!

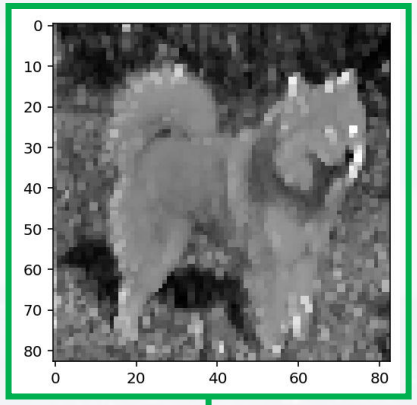
dinputs and mask have to look the same (but different values)!

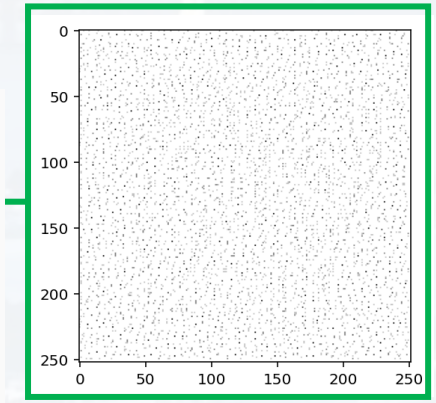
```
Conv1.forward(I,2,1)
MP1.forward(Conv1.output,3,5)
MP1.backward(MP1.output)

plt.imshow(Conv1.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(MP1.output[:,:,0,4], cmap = 'gray')
plt.show()
plt.imshow(MP1.dinputs[:,:,0,4], cmap = 'gray_r')
plt.show()
plt.imshow(MP1.mask[:,:,0,4], cmap = 'gray_r')
plt.show()
```

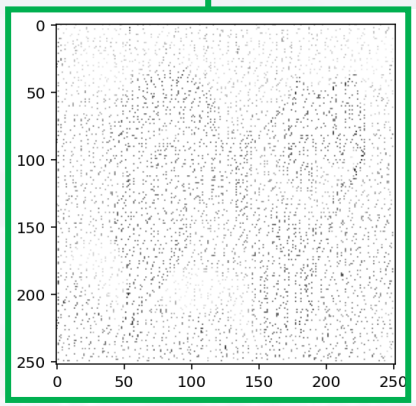
```
Diff = MP1.mask[:,:,0,4]/
       MP1.dinputs[:,:,0,
```

```
In [20]: Diff
Out[20]:
array([[ 0.,  0.,  0., ...,  0.,  0., -0.],
       [ 0.,  0.,  0., ...,  0.,  0., -0.],
       [ 0.,  0.,  0., ...,  0.,  0., -0.],
       ...,
       [ 0.,  0.,  0., ...,  0.,  0., -0.],
       [ 0.,  0.,  0., ...,  0.,  0., -0.],
       [ 0.,  0.,  0., ...,  0., -0., -0.]])
```









convolution

for **backpropagation**, we need to **up-sample** the images!

same for convolution layer now:

`backward` `dinputs = dvalues * weights` (we don't need for first conv layer!)

forward:

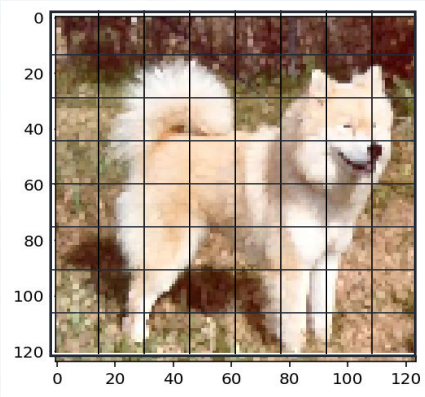
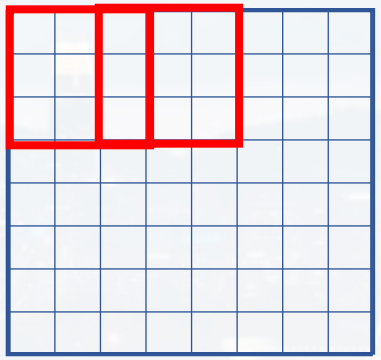
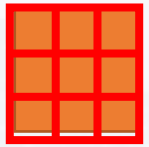


image = inputs

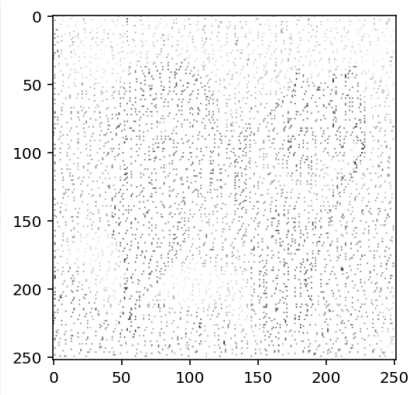


$$\sum_i I_i w_i + b$$



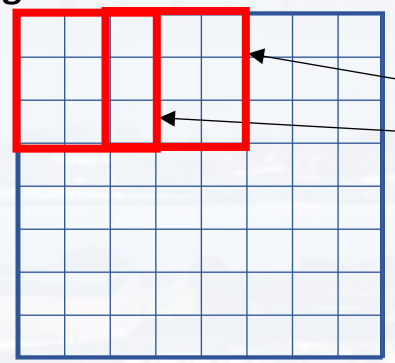
convolved image

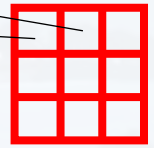
backward:



dinputs

weights





dvalues

convolution

for **backpropagation**, we need to **up-sample** the images!

same for convolution layer now:

backward **dinputs = dvalues * weights** (we don't need for first conv layer!)

forward:

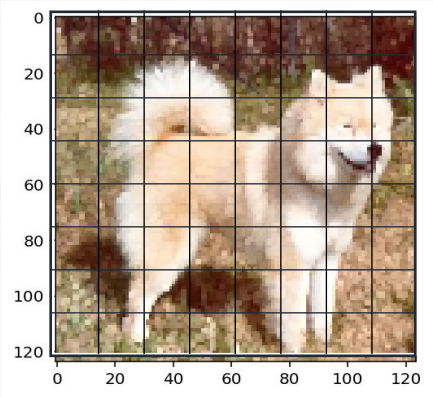
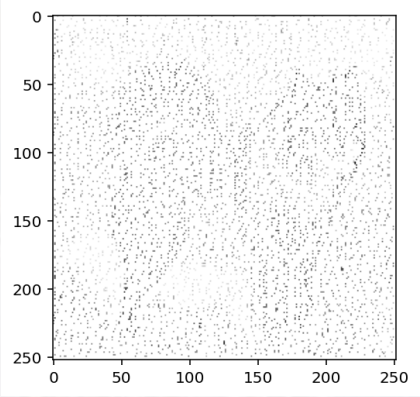


image = inpu

```
for i in range(numImds):# loop over number of images
    currentIm_pad = imagePadded[:,:,:,i]# Select ith padded image
    for k in range(NK):# loop over kernels (= #filters)
        for c in range(numChan):# loop over channels of incomming data
            if filt[c,k] ==1:
                for y in range(yd):# loop over axis of output
                    for x in range(xd):# loop over axis of output
                        # finding corners of the current "slice" (≈4 lines)
                        y_start = y*stride
                        y_end   = y_start + yK
                        x_start = x*stride
                        x_end   = x_start + xK
                        sx      = slice(x_start,x_end)
                        sy      = slice(y_start,y_end)
                        current_slice = currentIm_pad[sx,sy,c]

                        dweights[:,:,k]    += current_slice * dvalues[x,y,k,i]
                        dinputs[sx,sy,c,i] += weights[:,:,k]* dvalues[x,y,k,i]
        dbiases[0,k] += np.sum(np.sum(dvalues[:,:,k,i],axis=0),axis=0)
```

backward:



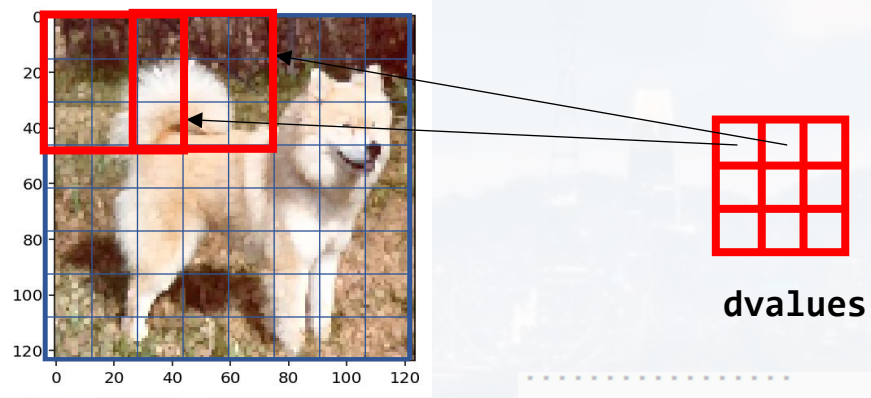
convolution

for **backpropagation**, we need to **up-sample** the images!

same for convolution layer now:

backward `dweights = dvalues * inputs`

backward:




```
                # finding corners of the current "slice" (≈4 lines)
                y_start = y*stride
                y_end   = y_start + yK
                x_start = x*stride
                x_end   = x_start + xK

                sx      = slice(x_start,x_end)
                sy      = slice(y_start,y_end)

                current_slice = currentIm_pad[sx,sy,c]
dweights[:,:,k]     += current_slice * dvalues[x,y,k,i]
                dinputs[sx,sy,c,i] += weights[:,:,k]* dvalues[x,y,k,i]

        dbiases[0,k] += np.sum(np.sum(dvalues[:,:,k,i],axis=0),axis=0)
```

convolution

for **backpropagation**, we need to **up-sample** the images!

same for convolution layer now:

backward **dbiases = dvalues**

backward:

```
                    # finding corners of the current "slice" (≈4 lines)
                    y_start = y*stride
                    y_end   = y_start + yK
                    x_start = x*stride
                    x_end   = x_start + xK

                    sx      = slice(x_start,x_end)
                    sy      = slice(y_start,y_end)

                    current_slice = currentIm_pad[sx,sy,c]

                    dweights[:,:,k]    += current_slice * dvalues[x,y,k,i]
                    dinputs[sx,sy,c,i] += weights[:,:,k]* dvalues[x,y,k,i]

        dbiases[0,k] += np.sum(np.sum(dvalues[:,:,k,i],axis=0),axis=0)
```

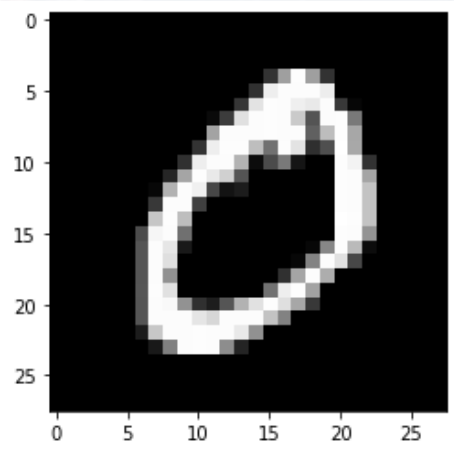
```
dbiases[0,k] += np.sum(np.sum(dvalues[:,:,k,i],axis=0),axis=0)
```

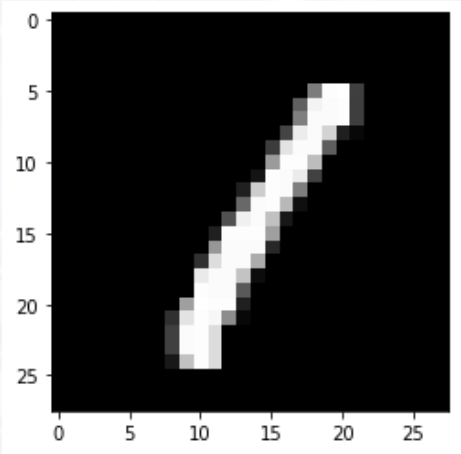
We are done now and can explore our self-made LeNet

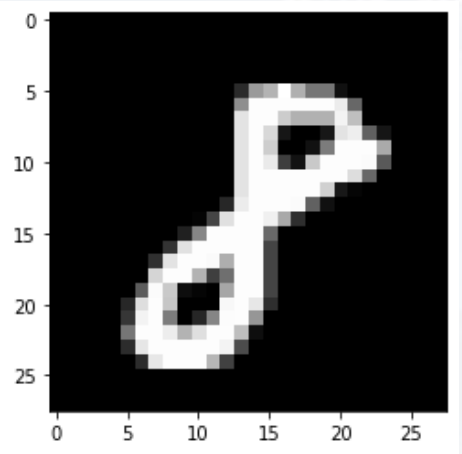
- MyCNN.py
  - Read_Scale_Imds
  - ConvLayer
    - __init__
    - forward
    - backward
  - Average_Pool
  - Min_Pool
  - Max_Pool
  - Sigmoid
  - Tanh
  - Activation_ReLU
  - Flat
  - Layer_Dense
  - Activation_Softmax
  - Loss
  - Loss_CategoricalCrossEntropy
  - CalcSoftmaxLossGrad
  - Optimizer_SGD

```
#pip install keras
#pip install tensorflow

from keras.datasets import mnist

(train_x, train_y), (test_x, test_y) = mnist.load_data()
```

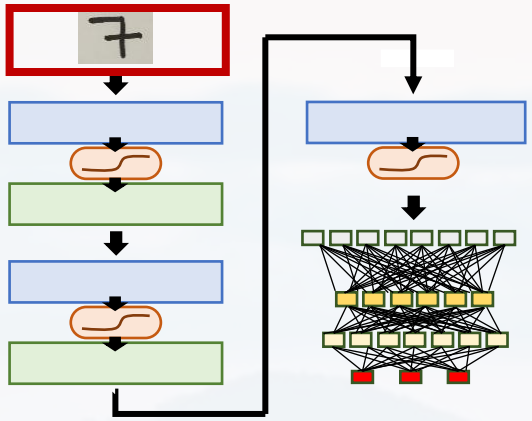
```
plt.imshow(train_x[1,:,:], cmap = 'gray')
```

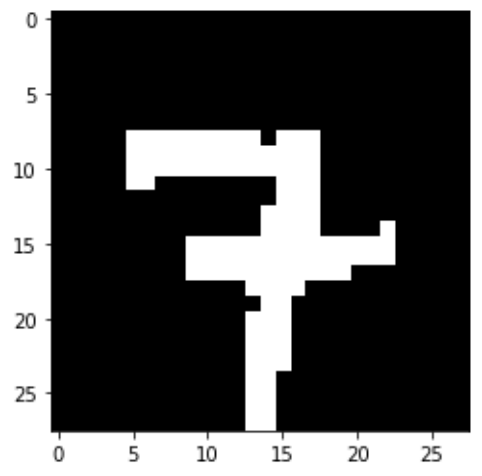




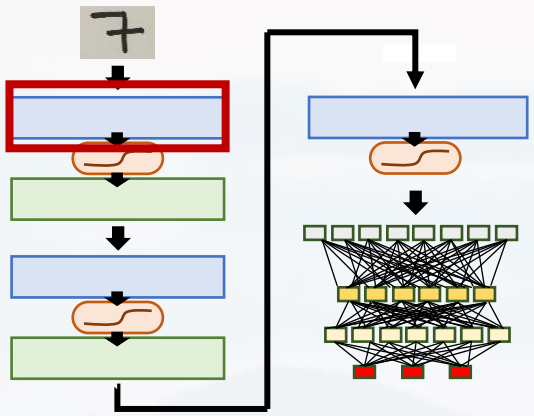


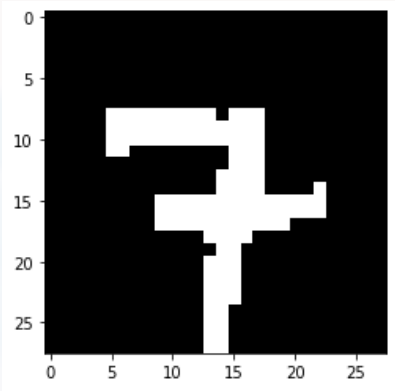
exploring self-made LeNet:

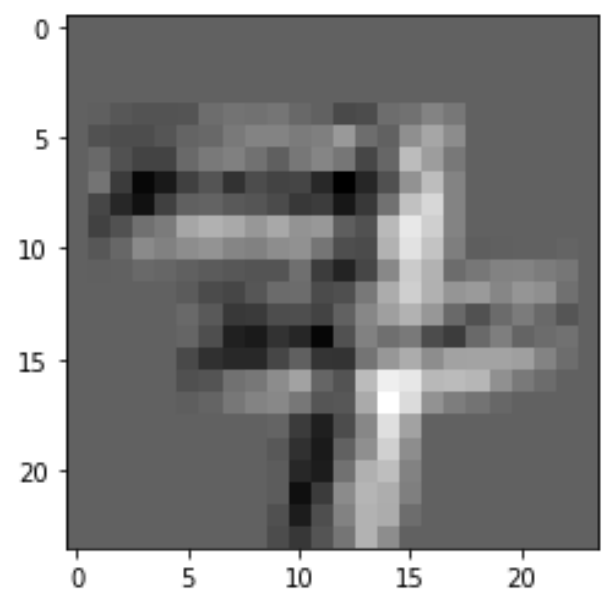




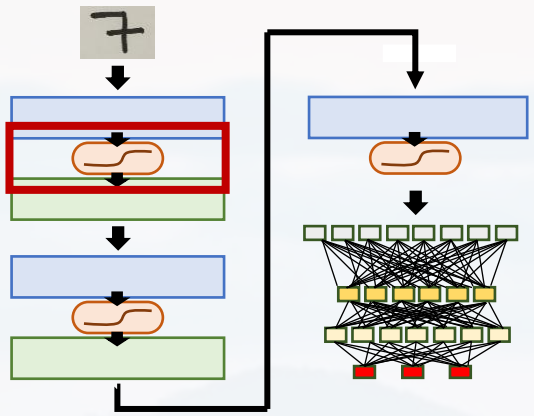
exploring self-made LeNet:

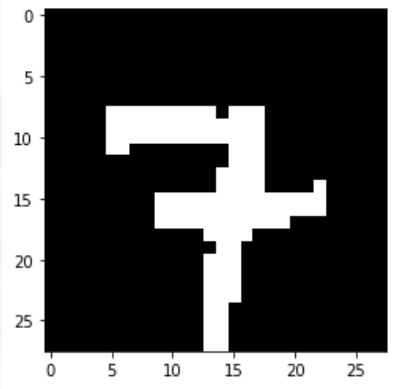


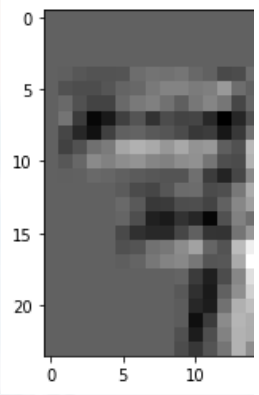


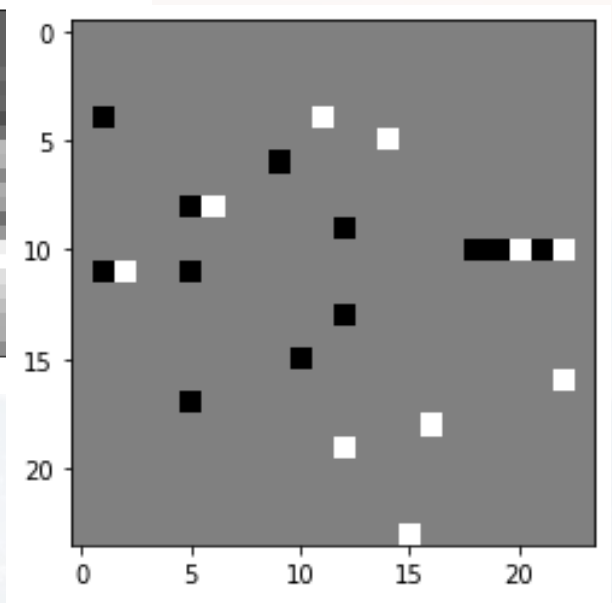


exploring self-made LeNet:

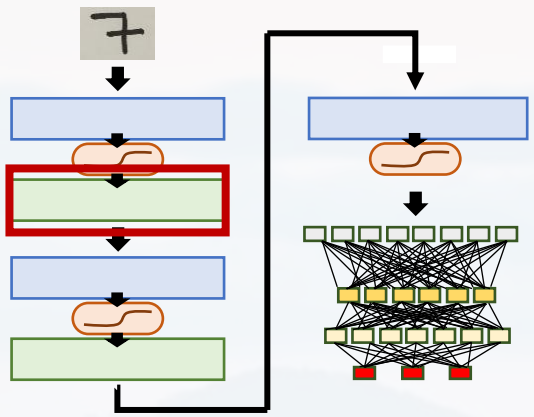


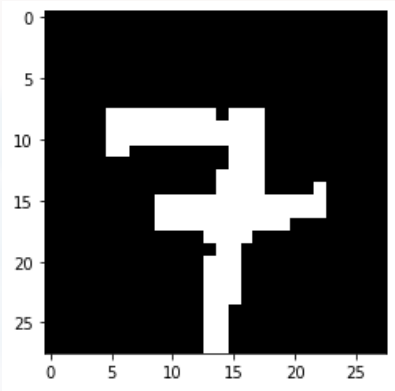


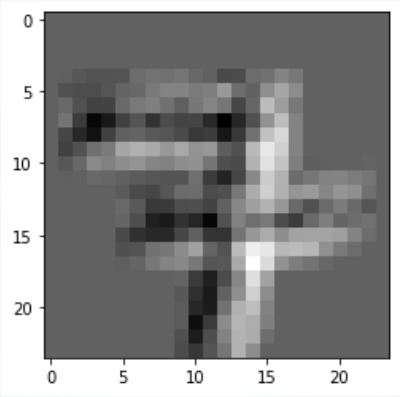


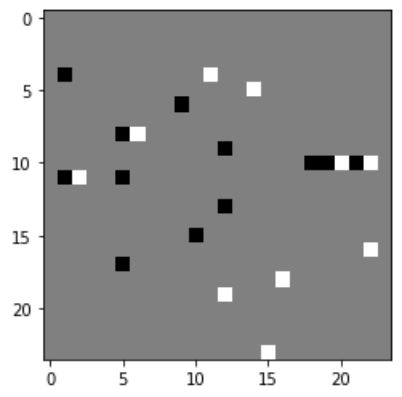


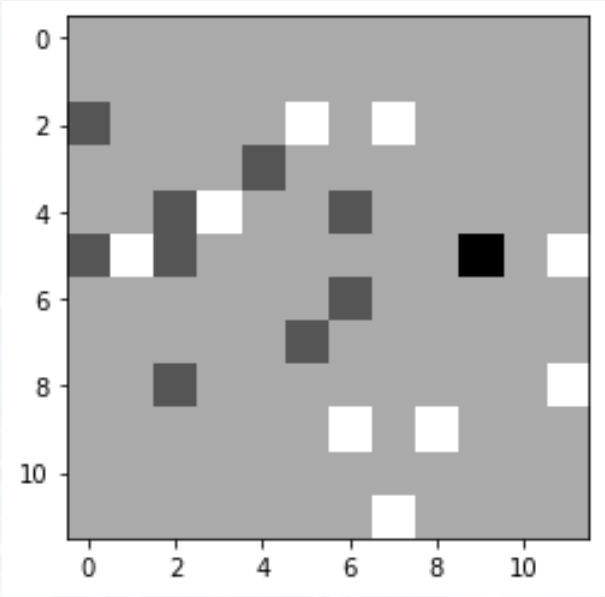
exploring self-made LeNet:



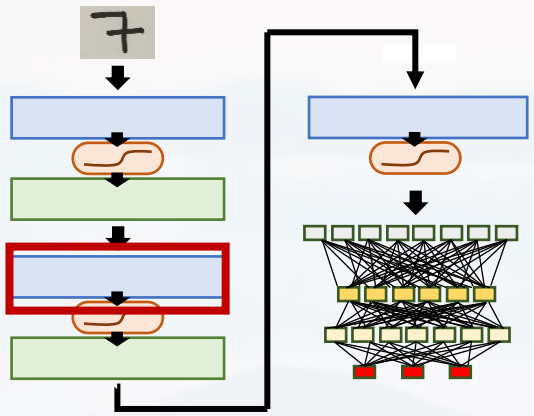


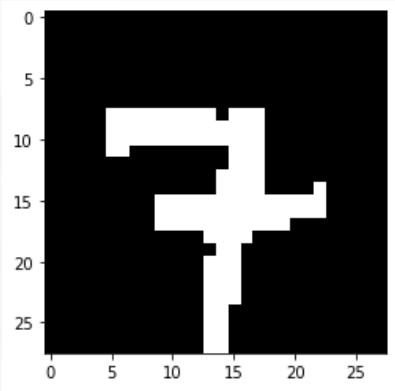


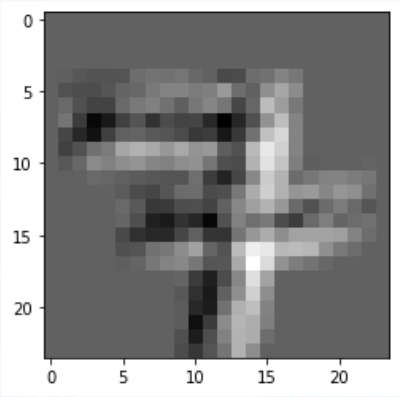


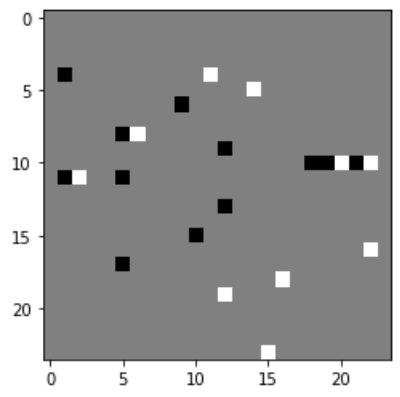


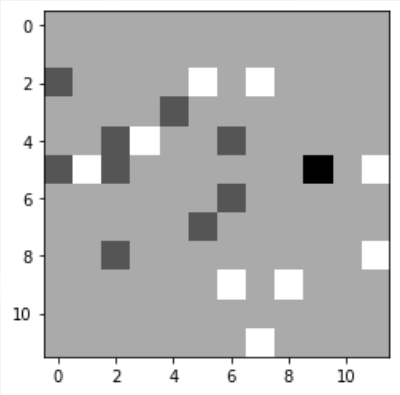
exploring self-made LeNet:

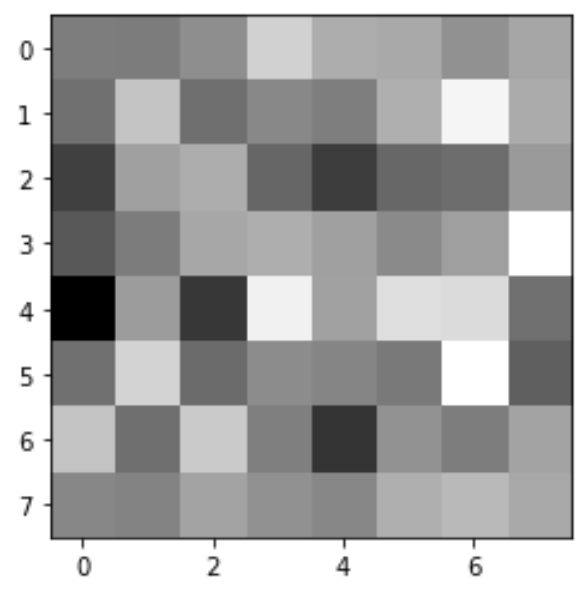




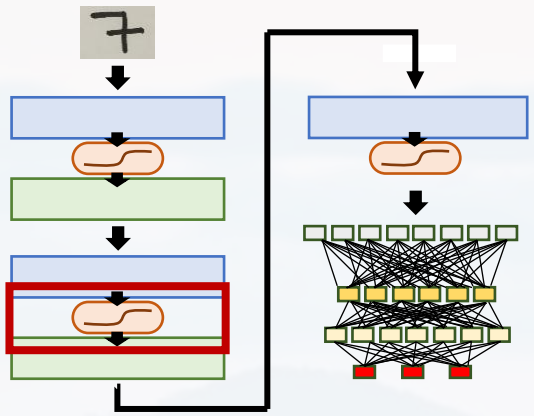


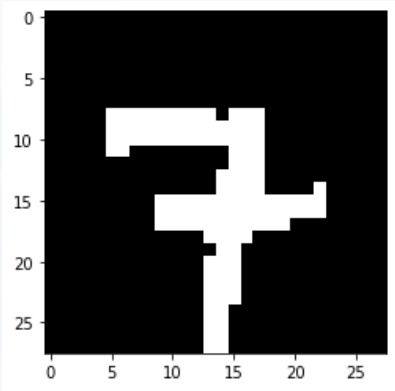


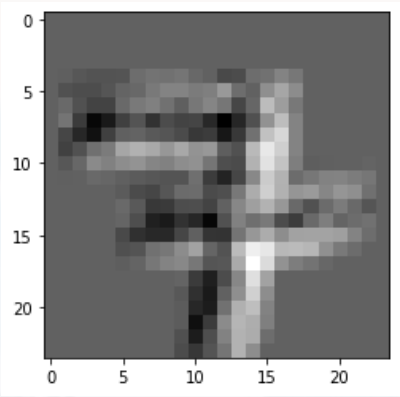


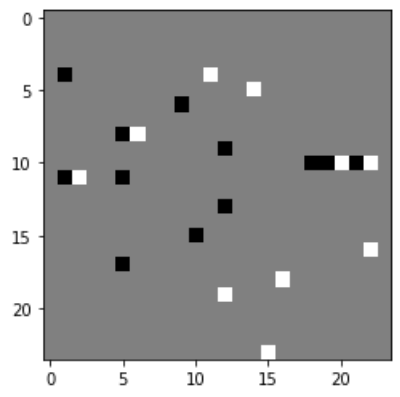


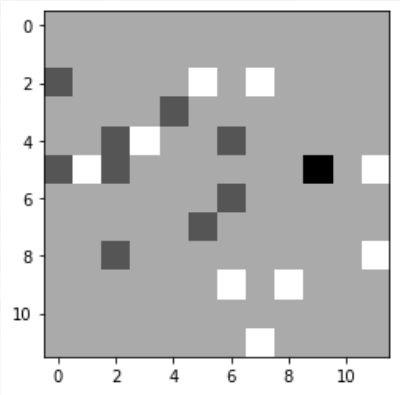
exploring self-made LeNet:

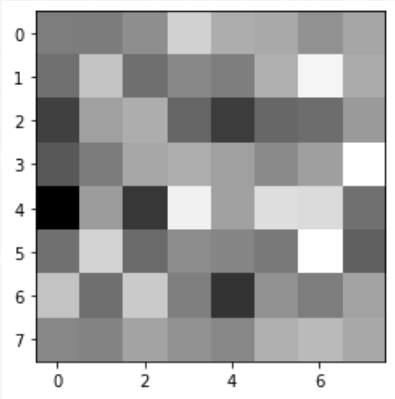


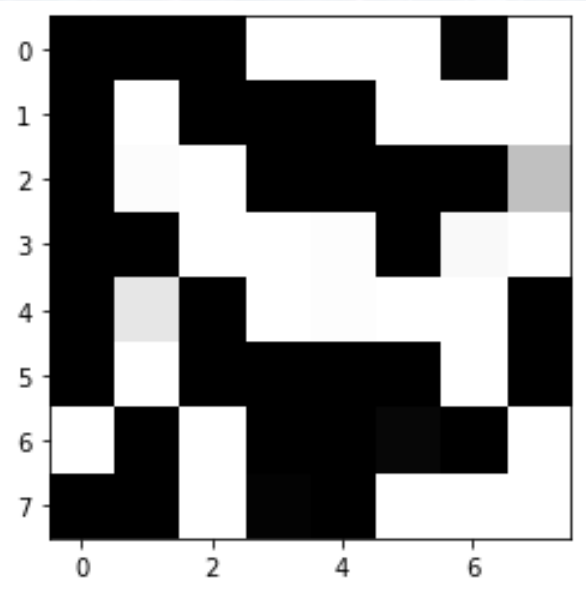




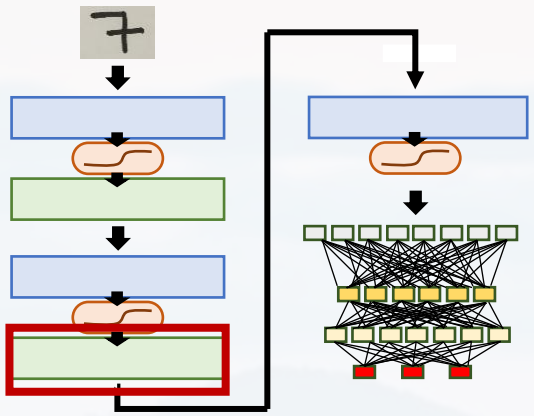


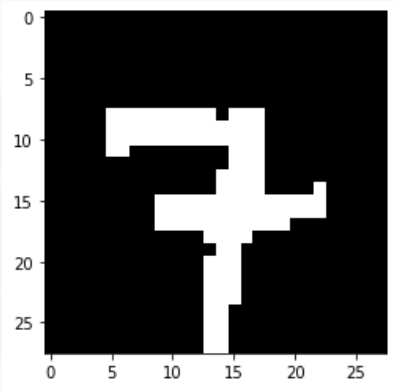


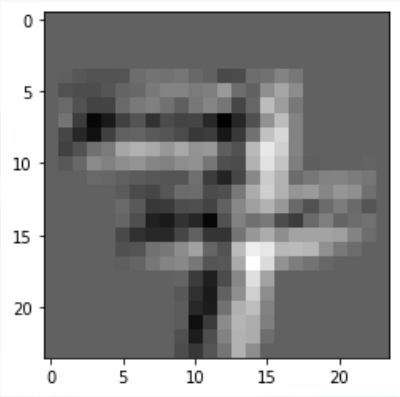


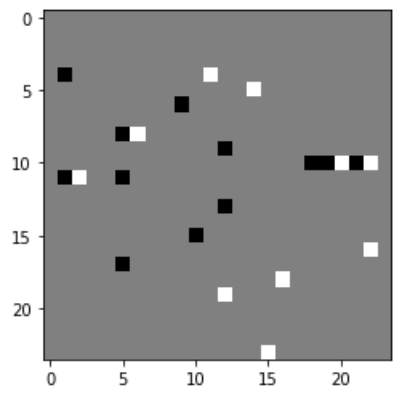


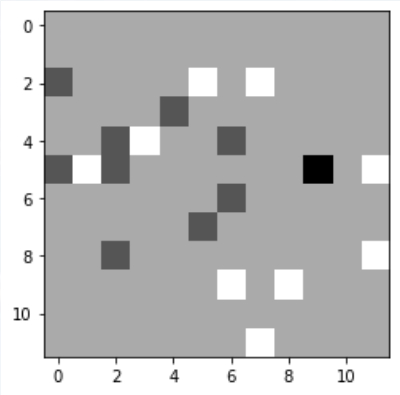
exploring self-made LeNet:

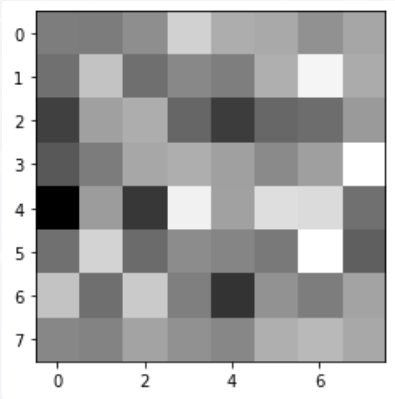


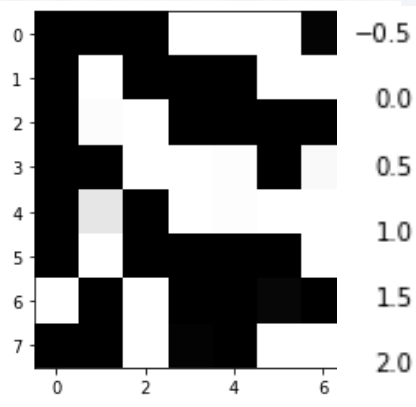


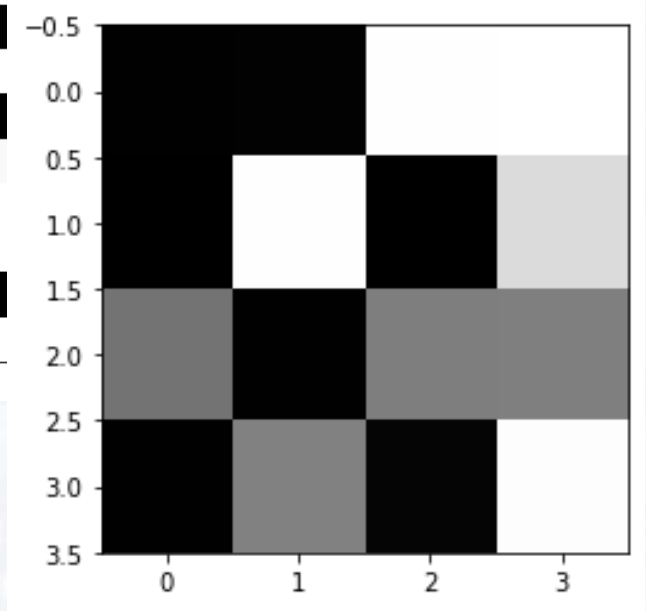




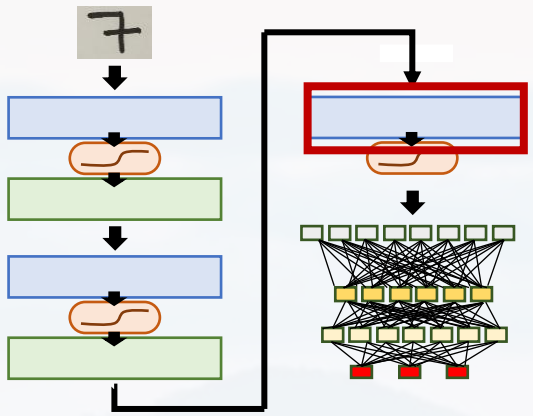


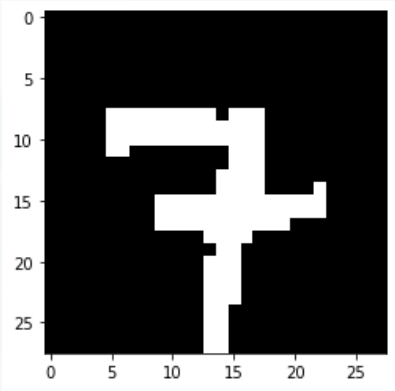


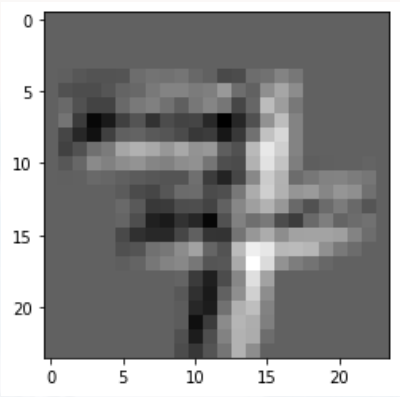


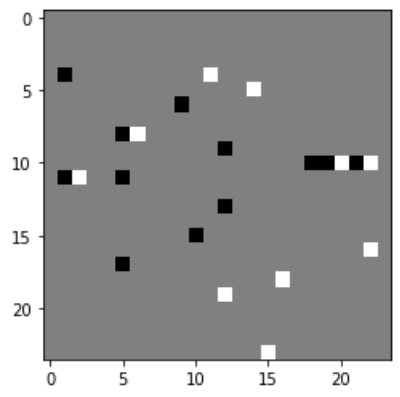


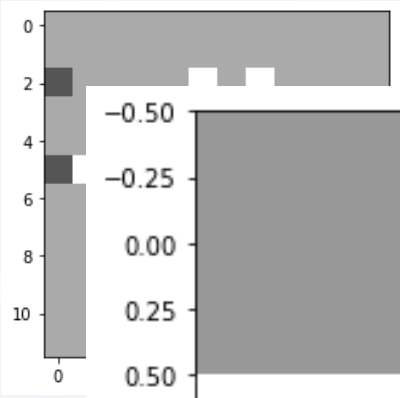
exploring self-made LeNet:

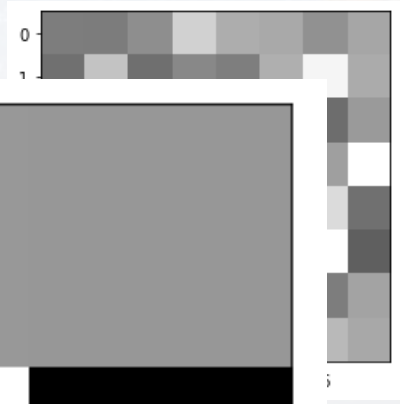


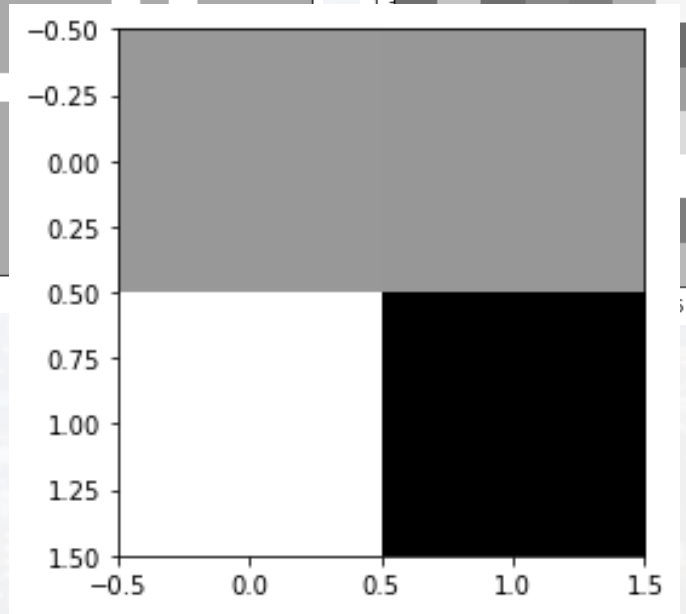


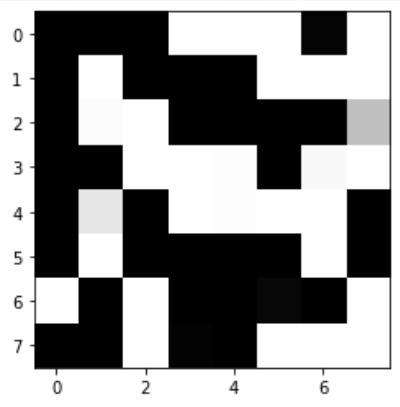


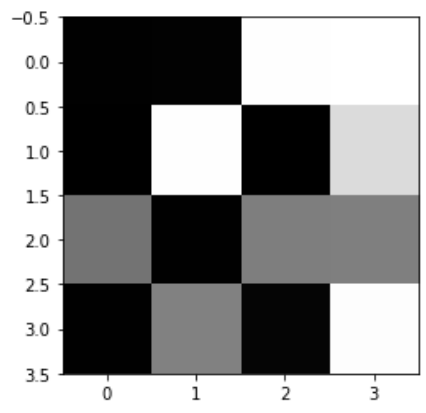




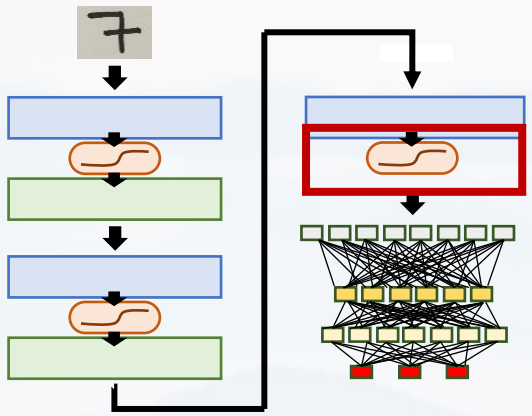


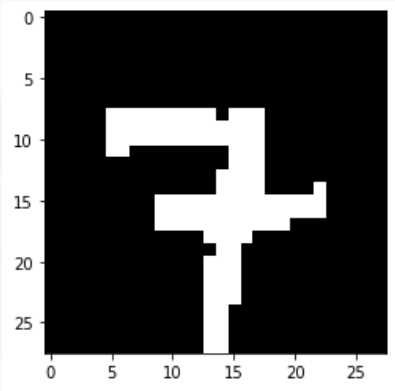


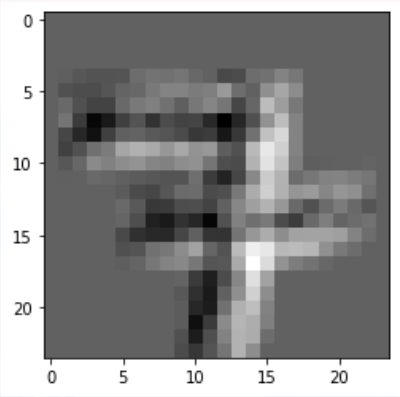


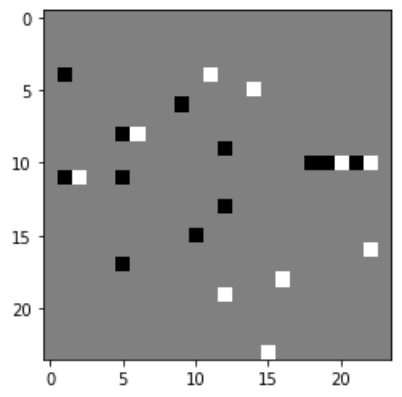


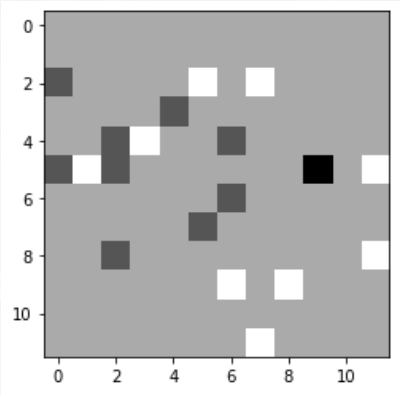
exploring self-made LeNet:

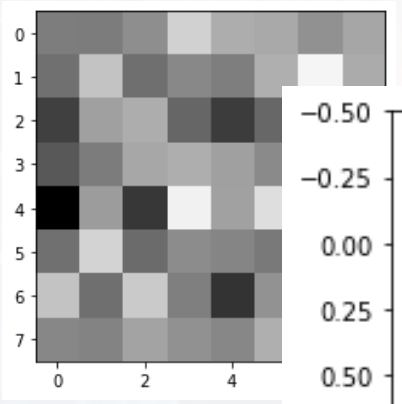


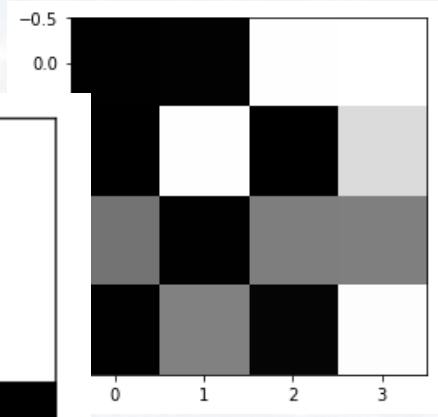


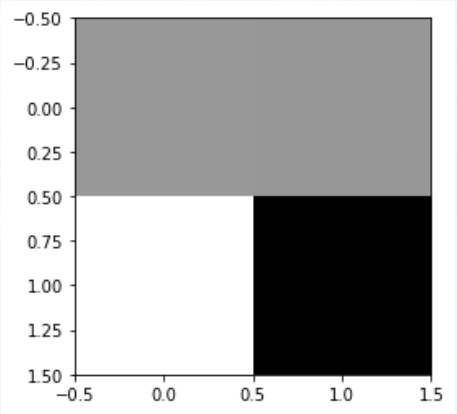


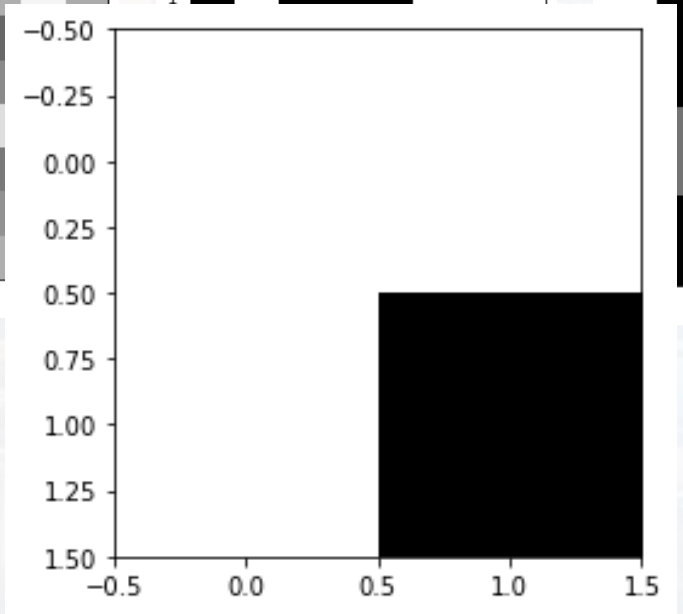




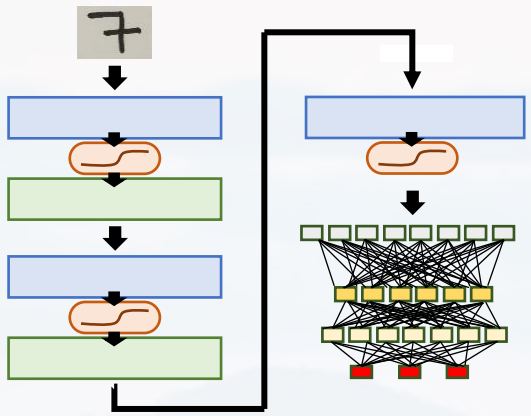


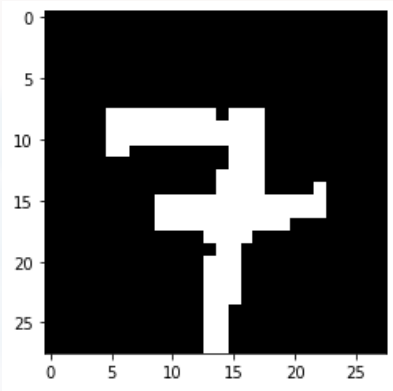


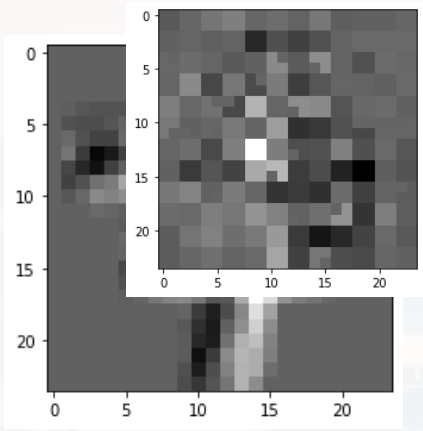


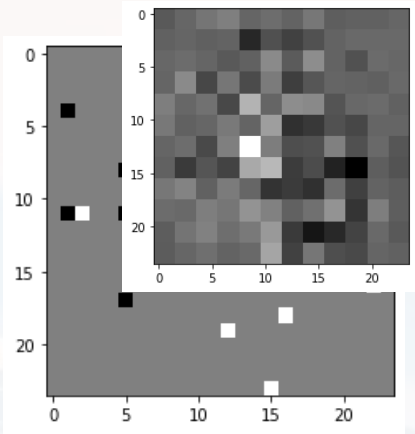


exploring self-made LeNet:

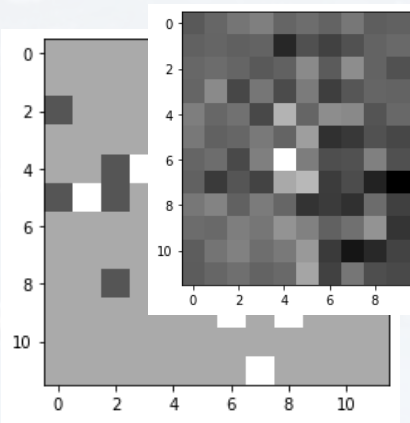


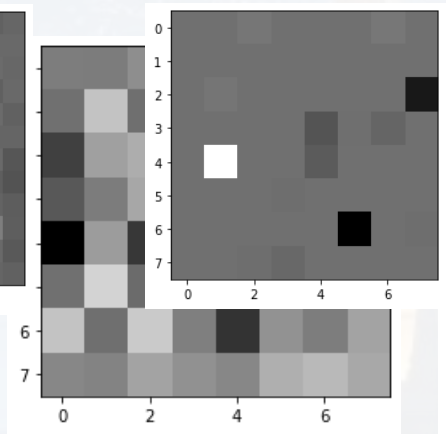


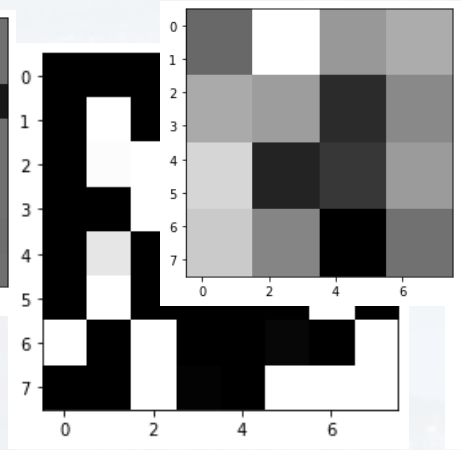


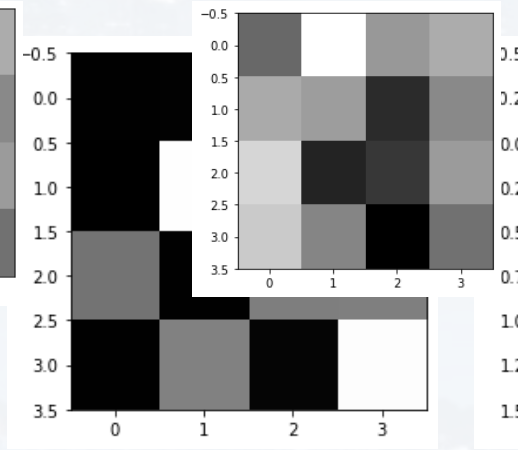


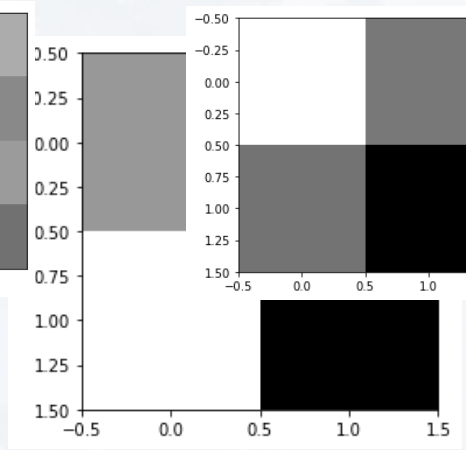
now back propagation!

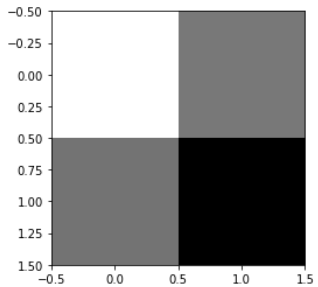


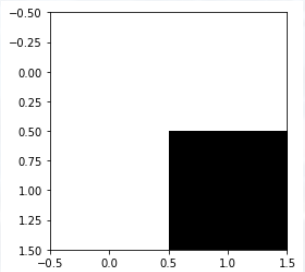


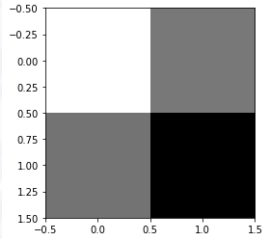










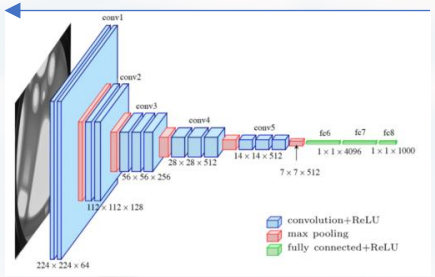


Backward Part Spelled Out

CNNs have an “hourglass” structure

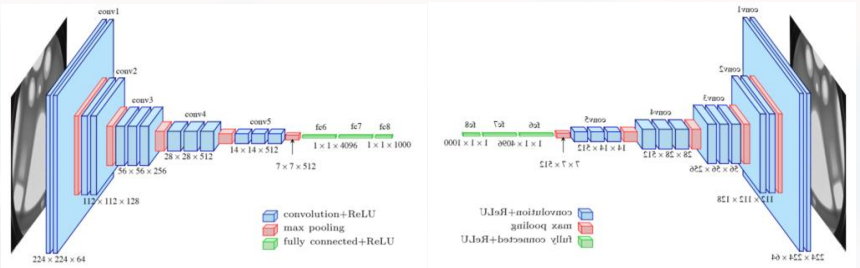
classification:

backward (up-sampling)



forward (down-sampling)

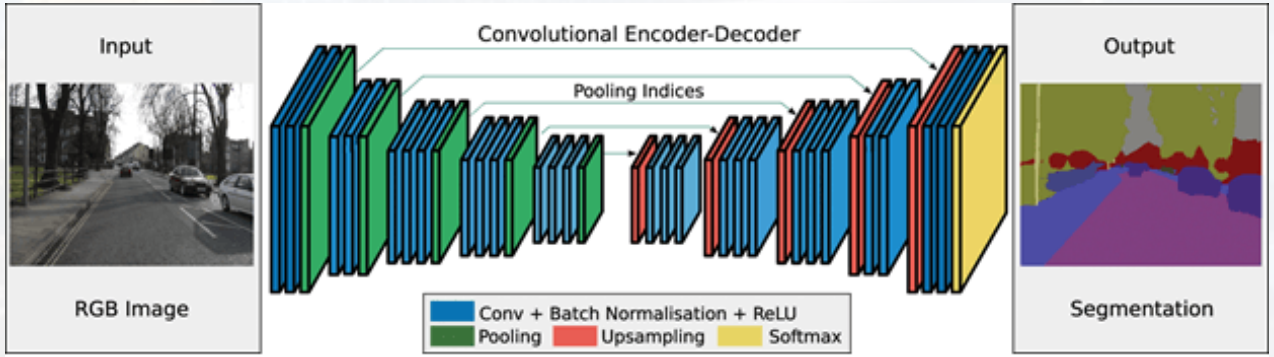
backward (up-sampling)



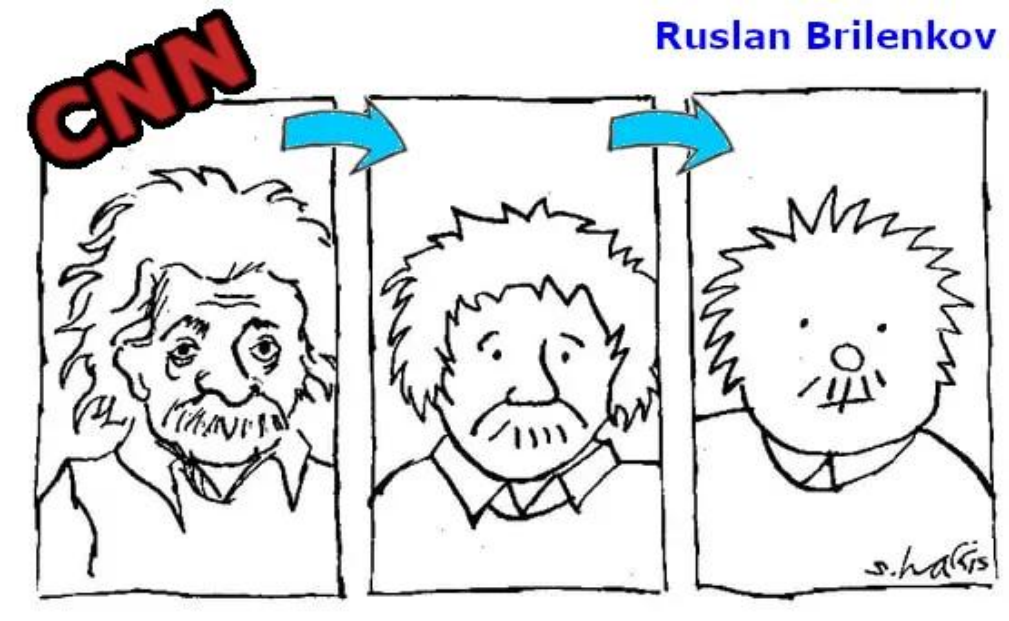
forward (down-sampling)

segmentation:

backward (up-sampling) ***backward (down-sampling)***



forward (down-sampling) ***forward (up-sampling)***

Discussion Module 10

Thank you very much for your attention!